

VLM-RL: A Unified Vision Language Models and Reinforcement Learning Framework for Safe Autonomous Driving

Zilin Huang^{†a}, Zihao Sheng^{†a}, Yansong Qu^{†b}, Junwei You^a, Sikai Chen^{a,*}

^aDepartment of Civil and Environmental Engineering, University of Wisconsin-Madison, Madison, WI, 53706, USA

^bLyles School of Civil Engineering, Purdue University, West Lafayette, IN 47907, USA

Abstract

In recent years, reinforcement learning (RL)-based methods for learning driving policies have gained increasing attention in the autonomous driving community and have achieved remarkable progress in various driving scenarios. However, traditional RL approaches **rely on manually engineered rewards**, which require extensive human effort and often lack generalizability. To address these limitations, we propose **VLM-RL**, a unified framework that integrates pre-trained **Vision-Language Models** (VLMs) with **RL** to **generate reward signals using image observation and natural language goals**. The core of VLM-RL is the **contrasting language goal (CLG)**-as-reward paradigm, which uses positive and negative language goals to generate semantic rewards. We further introduce a **hierarchical reward synthesis approach** that combines CLG-based semantic rewards with vehicle state information, improving **reward stability** and offering a **more comprehensive reward signal**. Additionally, a **batch-processing** technique is employed to optimize computational efficiency during training. Extensive experiments in the CARLA simulator demonstrate that VLM-RL outperforms state-of-the-art baselines, achieving a 10.5% reduction in collision rate, a 104.6% increase in route completion rate, and robust generalization to unseen driving scenarios. Furthermore, VLM-RL can seamlessly integrate almost any standard RL algorithms, potentially revolutionizing the existing RL paradigm that relies on manual reward engineering and enabling continuous performance improvements. The demo video and code can be accessed at: <https://zilin-huang.github.io/VLM-RL-website/>.

Keywords: Autonomous Driving, Vision Language Models, Reinforcement Learning, Reward Design

1. Introduction

Autonomous driving technology has made significant progress in recent years, yet achieving **human-level safety** and **reliability** remains a fundamental challenge (Feng et al., 2023; Huang et al., 2024b). A particularly acute challenge is how to develop safe and generalizable driving policies for complex traffic environments (Di and Shi, 2021; Cao et al., 2022; He et al., 2024; Huang et al., 2024d; Sheng et al., 2024a; Yao et al., 2023). The rapid advancement of deep learning has catalyzed remarkable developments in this domain, particularly through imitation learning (IL) and reinforcement learning (RL) (Huang et al., 2024c; Wu et al., 2024), as shown in Fig. 1 (a). IL aims to learn driving policies by mimicking expert demonstrations, achieving impressive performance in controlled environments (Huang et al., 2024c). Yet, IL-based methods face inherent limitations: they **heavily depend on the scale and quality of demonstration data** and exhibit poor generalization beyond the training distribution. In contrast, RL enables agents to actively learn optimal driving policies through direct interaction with the environment by maximizing carefully designed reward functions (Huang et al., 2024c). The effectiveness of RL-enabled methods has been demonstrated in various decision-making scenarios, such as safe navigation (Mao et al., 2024; He et al., 2024), car-following control (Hart et al., 2024), trajectory control (Sheng et al., 2024b), and lane change (Guo et al., 2024).

Nevertheless, a major challenge in applying RL is **designing an appropriate reward function** that will lead to the desired behavior (Ma et al., 2023; Venuto et al., 2024; Xie et al., 2024; Wang et al., 2024). While RL has been remarkably successful in domains where the reward function is clearly defined (e.g., gaming, robot manipulation), its application to autonomous driving remains troubled (Ye et al., 2024; Hazra et al., 2024; Han et al., 2024; Zhou et al., 2024). The fundamental difficulty is that the notion of “*good driving*” encompasses **complex, context-dependent behaviors**, **relying on tacit knowledge** that is difficult to quantify and encode as a reward function (Ye et al., 2024). This reflects Polanyi’s paradox, which asserts that “*we know more than we can tell*” (Polanyi, 2009). Traditionally, reward functions in the field of autonomous driving are usually manually designed based on **expert intuition** and **heuristics**, which often combine multiple sub-objectives such as speed maintenance, lane following, and collision avoidance (Chen et al., 2022; Wang et al., 2023a; Zhang et al., 2024). However, this procedure, known as “*reward*

*Corresponding author: Sikai Chen. † Equal contribution.

Email addresses: zilin.huang@wisc.edu (Zilin Huang[†]), zihao.sheng@wisc.edu (Zihao Sheng[†]), qu120@purdue.edu (Yansong Qu[†]), jyyou38@wisc.edu (Junwei You), sikai.chen@wisc.edu (Sikai Chen)

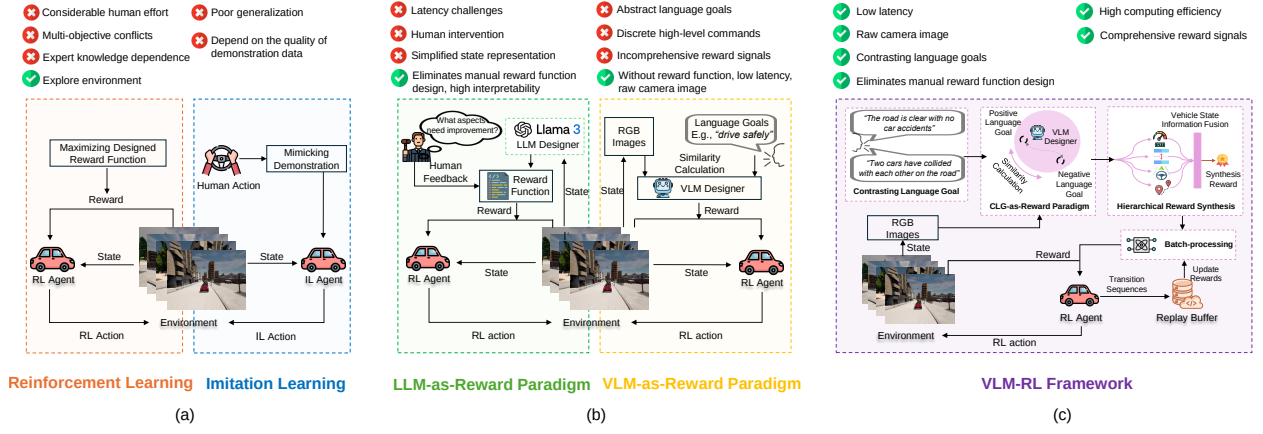


Figure 1: Comparative Overview of Reward Design Paradigms for Autonomous Driving. (a) Fundamentals and limitations of IL/RL-based methods for driving policy learning. (b) Fundamentals and limitations of foundation model-based reward design methods (i.e., LLM-as-Reward and VLM-as-Reward paradigms) for driving policy learning. (c) Our proposed VLM-RL framework, leverages VLMs to achieve a comprehensive and stable reward design for safe autonomous driving.

engineering”, requires **considerable human effort** and **trial-and-error iterations** (Abouelazm et al., 2024). As noted by Han et al. (2024); Knox et al. (2023); Abouelazm et al. (2024), it faces several challenges such as expert knowledge dependence, multi-objective conflicts, and generalizability limitations.

Recent breakthroughs in foundation models, particularly large language models (LLMs) (OpenAI, 2023) and vision-language models (VLMs) (Radford et al., 2021), have demonstrated remarkable capabilities in understanding natural language instructions and complex visual scenes. This progress has inspired researchers to explore the use of these foundation models for reward shaping in RL, offering a promising solution to the longstanding challenge of reward design. The key idea is to leverage the **rich semantic understanding capabilities of foundation models** to **translate human-specified goals into reward signals** that can guide RL training effectively (Venuto et al., 2024; Ma et al., 2023; Xie et al., 2024). In the field of robotics, integrating foundation models into reward functions has shown strong performance and promising generalization capabilities. Many experiments in this domain focus on tasks where the desired goal state is well-defined and easily specified, such as “*Put carrot in bowl*” for manipulation tasks (Kim et al., 2024). This explicit goal specification allows researchers to harness pre-trained foundation models **as zero-shot reward generators, simplifying the reward design process** (Baumli et al., 2023; Rocamonde et al., 2024; Sontakke et al., 2024; Fu et al., 2024). However, while these approaches perform well across various robotic tasks, they encounter significant challenges in the context of safe autonomous driving. Unlike robotic manipulation tasks, where goals can be specified with high precision, driving objectives such as “*drive safely*” are inherently abstract and context-dependent. These high-level instructions are difficult for foundation models to interpret effectively due to the **complexity** and **variability** of real-world driving scenarios (Ye et al., 2024).

Several recent works have attempted to combine foundation models with RL for autonomous driving. Fig. 1 (b) illustrates two dominant paradigms for this integration: *LLM-as-Reward* and *VLM-as-Reward* paradigm. The first approach directly invokes LLMs to generate reward functions or codes, incorporating human feedback to iteratively refine the design. The second approach uses VLMs as reward functions, where the model evaluates the agent’s state and provides immediate feedback based on the alignment with specified goals to guide behavior. Despite these advancements, existing works still face several critical limitations: (a) Most works rely solely on language descriptions to achieve desired behaviors (Zhou et al., 2024; Yildirim et al., 2024). While Ye et al. (2024) proposes an opposite reward design, it focuses exclusively on negative scenarios and overlooks the rich semantic relationships between positive and negative driving behaviors. (b) Many approaches depend on real-time foundation model inference during deployment (Hazra et al., 2024; Han et al., 2024; Zhou et al., 2024), introducing latency issues that are unacceptable for safety-critical driving decisions where real-time responsiveness is essential. (c) Current methods predominantly validate their effectiveness in simplified simulation environments such as HighwayEnv simulator (Yildirim et al., 2024; Ye et al., 2024; Han et al., 2024; Zhou et al., 2024). These methods use simplified state representations, which do not capture the complexity of real-world sensors (e.g., camera images) used in actual autonomous vehicles. (d) Many existing approaches generate discrete high-level commands (e.g., lane changes, acceleration) (Zhou et al., 2024; Ye et al., 2024), which are insufficient for continuous and precise control needed in real-world vehicle operations.

Observing how humans learn new skills, we find that people typically learn more effectively through contrasting examples. For instance, when teaching someone to cook a steak, instructors often highlight both the correct and incorrect techniques: “*A perfectly cooked steak has a golden-brown crust and a uniformly pink interior*” versus “*If the steak turns completely dark brown and has a burnt smell, it is overcooked*”. This **helps learners develop a comprehensive understanding** of proper cooking techniques by recognizing both desired and undesired outcomes. Drawing inspiration from human learning, we propose a unified framework for integrating pre-trained VLMs and online RL, named **VLM-RL**, as shown in Fig. 1 (c). VLM-RL fundamentally rethinks how foundation models can be integrated

into RL-based autonomous driving systems. We first introduce the concept of contrasting language goal (CLG) to guide RL-based safe driving tasks. Building upon the *VLM-as-Reward*, a novel *CLG-as-Reward* paradigm is then presented, leveraging CLG to generate more informative and context-aware rewards. To enhance learning stability, a hierarchical reward synthesis approach is adopted, combining CLG-based rewards with vehicle state information. These synthesized rewards are then integrated with standard RL for policy training. Additionally, a batch-processing technique is employed to improve computational efficiency during the training process.

More importantly, VLM-RL implements a closed-loop end-to-end training pipeline that integrates camera-based perception with continuous control outputs, addressing a significant limitation of existing work that typically relies on simplified state representations and discrete actions. Our contributions can be summarized as follows:

- We propose the VLM-RL framework, which leverages pre-trained VLMs as zero-shot reward models, eliminating the need for explicit reward engineering in RL-based safe driving tasks. **To our knowledge, VLM-RL is the first work in the autonomous driving field to unify VLMs with RL for end-to-end driving policy learning in the CARLA simulator.**
- We propose a novel CLG-as-Reward paradigm for reward shaping, which leverages pre-trained VLMs to generate semantic reward signals by measuring the semantic alignment between driving states and contrasting language descriptions (i.e., positive and negative language goals).
- We propose a hierarchical reward synthesis approach that combines CLG-based rewards with vehicle state information to provide comprehensive and stable reward signals. This method addresses the limitations of using only coarse semantic rewards from VLMs, which can mislead policy optimization due to incomplete or imprecise behavior descriptions.
- We develop a batch-processing technique to ensure computational efficiency. Instead of calculating rewards immediately for every observation, batches of observations are periodically sampled from a replay buffer and processed through the pre-trained VLM. VLM-RL can be seamlessly integrated into almost any standard RL, enabling consistent performance improvements.
- We conducted extensive experiments in the CARLA simulator, demonstrating significant improvements in safety, efficiency, and generalization to diverse driving scenarios. Specifically, compared to state-of-the-art baselines, VLM-RL achieved 10.5% reduction in collision rate, a 104.6% increase in route completion rate, and successfully generalized to previously unseen scenarios without fine-tuning.

The remainder of this paper is organized as follows. Section 2 reviews the related work. Section 3 introduces the preliminaries and problem formulation. Section 4 details the proposed VLM-RL framework. Section 5 presents the experimental setup and results. Finally, Section 6 concludes the paper and outlines future research directions.

2. Related Works

2.1. Reward Design with Foundation Model

The design of reward functions remains a fundamental challenge in RL. Recently, a new paradigm has emerged that **leverages foundation models to generate reward signals for RL**. Kwon et al. (2023) first demonstrated the potential of LLMs, such as GPT-3 (OpenAI, 2023), in generating rewards for text-based tasks. Subsequent works extended this idea, showing that LLMs can generate structured code for robot training (Yu et al., 2023) and Python code for various agents (Xie et al., 2024; Ma et al., 2023). However, these methods often assume **access to detailed environment information**, which is challenging in autonomous driving. For instance, accurate data on surrounding vehicles’ velocities and positions may not be available. In this work, the VLM-RL generates the reward signal **directly from the visual input captured by the on-board camera**, which does not require such assumptions. VLM-CaR (Venuto et al., 2024) mitigates VLM query costs by breaking tasks into sub-objectives, though this is difficult for safe driving tasks. Other works use the embedding space of pre-trained VLMs, such as CLIP (Radford et al., 2021). Mahmoudieh et al. (2022) are the first to use fine-tuned CLIP as reward models for robotic manipulation. VLM-SR (Baumli et al., 2023) converts similarity-based rewards into binary rewards via thresholding, while RoboCLIP (Sontakke et al., 2024) compares task video embeddings to agent behavior. VLM-RM (Rocamonde et al., 2024) enhances rewards using goal-based baseline regularization, and RL-VLM-F (Wang et al., 2024) incorporates human preference labels for improved reward quality, and FuRL (Fu et al., 2024) addresses reward misalignment issues to refine reward signals further. These methods work well in robotics domains where goal states can be precisely defined and easily understood by VLMs. In contrast, autonomous driving involves inherently ambiguous language goal states that are difficult to define or verify. Additionally, robotics tasks typically involve static or controlled environments, whereas autonomous driving must deal with **multiple agents** and **uncertain dynamic** scenarios.

2.2. Foundation Model in Autonomous Driving

Recent breakthroughs in foundation models have inspired researchers to apply them to the field of autonomous driving, including scene understanding (e.g., DriveVLM (Tian et al., 2024), LeapAD (Mei et al., 2024)), planning (e.g., DiLu (Wen et al., 2023), DriveMLM (Wang et al., 2023b)), scene generation (e.g., ChatScene Zhang et al. (2024), ChatSim (Wei et al., 2024)), human-vehicle interaction (e.g., DriVLM (Huang et al., 2024a), Drive as you speak (Cui et al., 2024)), and end-to-end driving (e.g., LMDrive (Shao et al., 2024), DriveGPT4 (Xu et al., 2024)). Despite these advancements, leveraging foundation models for reward design in safe driving tasks has yet to be fully explored. LLM-RL (Zhou et al., 2024) employs LLMs to intuitively shape reward functions via natural language prompts, enabling more human-like driving behavior. HighwayLLM (Yildirim et al., 2024) integrates LLMs with RL to provide explainable decision-making in highway driving scenarios. REvolve (Hazra et al., 2024) frames reward design as an evolutionary search problem, **leveraging LLMs and human feedback** to create **human-aligned reward functions**. In contrast, VLM-RL does not require human feedback. AutoReward (Han et al., 2024) utilizes LLMs to generate and refine reward functions through a **closed-loop framework** automatically. Most of the existing works heavily rely on real-time inference from foundation models, which may raise limitations such as **latency issues**. VLM-RL does not rely on direct model queries but instead **utilizes their embedding spaces** for reward computation. Perhaps the work closest to ours is LORD (Ye et al., 2024), which uses **undesired language goals to shape agent behavior**. The key differences between VLM-RL and LORD are: (a) VLM-RL **uses both desired and undesired goals** combined with vehicle state information for richer reward signals; (b) VLM-RL uses **camera-based visual inputs for more realistic perception**; and (c) VLM-RL implements an **end-to-end pipeline that produces continuous control outputs**.

3. Preliminaries

3.1. Partially Observable Markov Decision Processes

A partially observable Markov decision process (POMDP) is defined by the tuple $(S, \mathcal{A}, \theta, R, \mathcal{O}, \phi, \gamma, d_0)$, where S is the state space, $\mathcal{A}$ is the action space, $\theta(s' | s, a) : S \times S \times \mathcal{A} \rightarrow [0, 1]$ represents the transition function, $R(s, a, s') : S \times \mathcal{A} \times S \rightarrow \mathbb{R}$ is the reward function, $\mathcal{O}$ denotes the observation space, $\phi(o | s) : S \rightarrow \Delta(\mathcal{O})$ is the observation distribution, and $d_0(s) : S \rightarrow [0, 1]$ is the initial state distribution. At each timestep, the environment occupies a state $s \in S$, and the agent selects an action $a \in \mathcal{A}$. The environment transitions to a new state s' with probability $\theta(s' | s, a)$. The agent then receives an observation o with probability $\phi(o | s')$ and a reward $r = R(s, a, s')$. A sequence of states and actions forms a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$, where $s_i \in S$ and $a_i \in \mathcal{A}$. The return for a trajectory τ is the discounted sum of rewards: $g(\tau; R) = \sum_{t=0}^T \gamma^t R(s_t, a_t, s_{t+1})$. The agent’s objective is to find a policy $\pi(a | s)$ that maximizes the expected return $G(\pi) = \mathbb{E}_\pi[g(\tau(\pi); R)]$.

3.2. Vision-Language Models

VLMs have seen significant advancements in recent years (You et al., 2024). These models are broadly defined as those capable of handling sequences of both **language inputs** $l \in \mathcal{L}^{\leq n}$ and **vision inputs** $i \in \mathcal{I}^{\leq m}$. In this context, $\mathcal{L}$ represents a finite alphabet, and $\mathcal{L}^{\leq n}$ refers to **strings of length up to n** . Similarly, $\mathcal{I}$ denotes the space of 2D RGB images, and $\mathcal{I}^{\leq m}$ consists of **image sequences of length up to m** . A notable class of pre-trained VLMs is CLIP (Radford et al., 2021), which includes a **language encoder CLIP_L** : $\mathcal{L}^{\leq n} \rightarrow \mathcal{V}$ and an **image encoder CLIP_I** : $\mathcal{I} \rightarrow \mathcal{V}$, both mapping to a shared latent space $\mathcal{V} \subseteq \mathbb{R}^k$. These encoders are trained jointly through contrastive learning on **image-caption pairs**. The training objective is to minimize the cosine distance between embeddings of matching pairs while maximizing it for non-matching pairs. CLIP has demonstrated strong performance in various **downstream tasks** and exhibits impressive **zero-shot transfer capabilities** (Rocamonde et al., 2024).

3.3. Problem Statement

We model the task of training an autonomous driving agent as a POMDP, similar to Rocamonde et al. (2024). The agent’s objective is to learn an optimal policy $\pi : S \rightarrow \mathcal{A}$ that maximizes the expected cumulative reward, expressed as $H(\pi) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \mid \pi \right]$. A key challenge in this context is to design an effective reward function $R(s, a, s')$ that guides the agent toward desirable behaviors. Traditional reward engineering requires manual specification of complex behaviors and constraints, which can be tedious, error-prone, and hard to generalize across diverse driving scenarios. Ideally, we wish to directly use VLMs to provide agents with rewards $R(s)$ to guide desired behaviors, as is done in the robotics domain. However, as mentioned earlier, using VLMs directly as rewards for autonomous driving still faces critical challenges. Our goal is **to create a specialized VLM-as-Reward framework for the safe driving task to eliminate the need for explicit reward functions** $G(\pi) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R_{\text{VLM}}(s) \mid \pi \right]$.

4. Framework: VLM-RL

In this section, we present a detailed description of VLM-RL framework. The framework addresses the fundamental challenge of reward design in autonomous driving by leveraging the semantic understanding capabilities of pre-trained VLMs (i.e., CLIP).

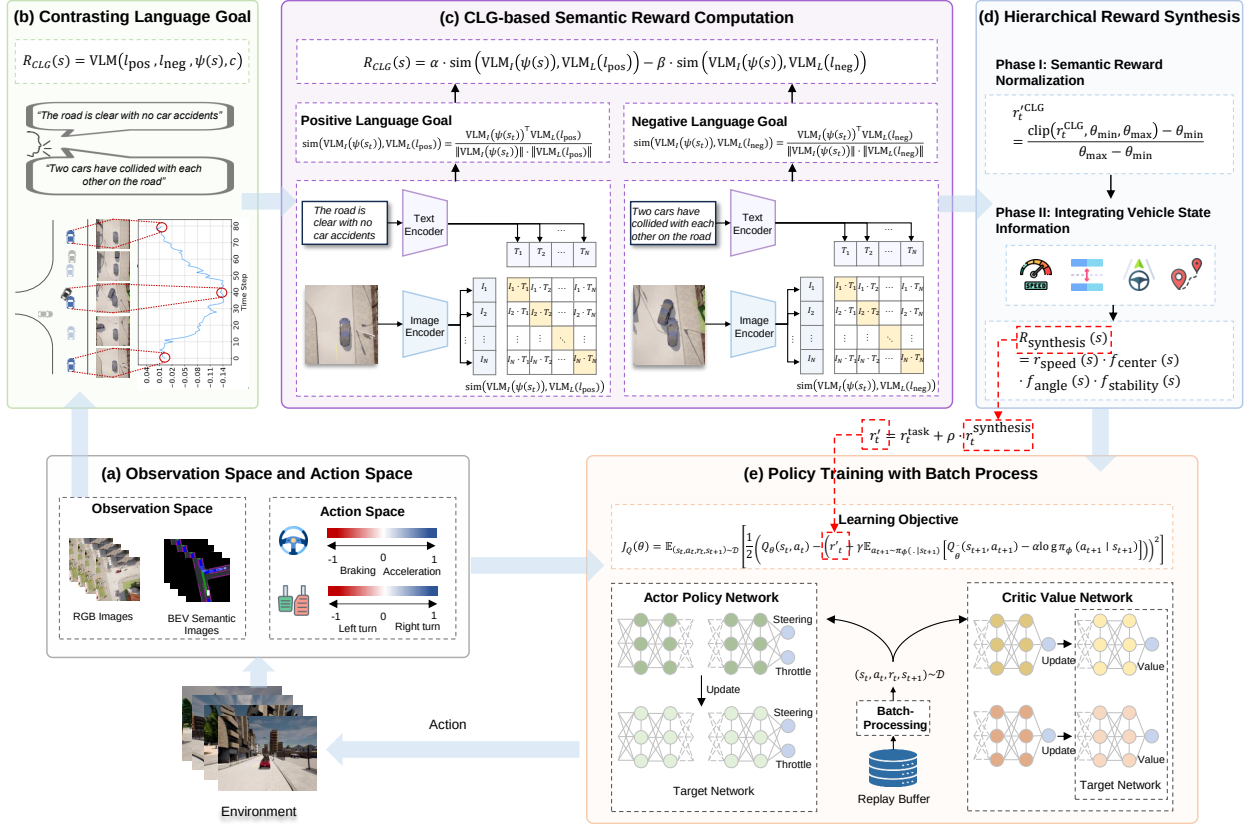


Figure 2: Architecture of the VLM-RL Framework for Autonomous Driving. (a) Observation and action spaces for policy learning; (b) Definition of CLG to provide semantic guidance; (c) CLG-based semantic reward computation using pre-trained VLMs; (d) Hierarchical reward synthesis that integrates semantic rewards with vehicle state information for comprehensive and stable reward signals; (e) Policy training with batch-processing, where SAC updates are performed using experiences stored in a replay buffer and rewards are computed asynchronously to optimize efficiency.

4.1. Overview

The VLM-RL framework consists of four main components. First, we define the concept of CLG that describes both desired and undesired driving behaviors, providing a foundation for reward computation. Second, we utilize CLIP to compute semantic alignment between the current driving state and these contrasting language descriptions, generating semantic reward signals. Third, we develop a hierarchical reward synthesis approach that combines the semantic rewards with vehicle state information (e.g., speed, heading angle) to produce stable and comprehensive reward signals. Fourth, to optimize computational efficiency, we implement a batch-processing technique that periodically processes observations from the replay buffer rather than computing rewards in real time. Fig. 2 illustrates the overall architecture of our framework. We describe each component in detail in the following subsections.

4.2. Contrasting Language Goal Definition

Recent advances in robotics have demonstrated remarkable success in utilizing pre-trained VLMs as zero-shot reward models across diverse tasks (Sontakke et al., 2024). Given a task $\mathcal{T}$ and its natural language description $l \in \mathcal{L}^n$, the fundamental approach involves leveraging VLMs to generate reward signals that guide the agent toward desired behaviors. This can be formally expressed as (Rocamonde et al., 2024)

$$R_{\text{VLM}}(s) = \text{VLM}(l, \psi(s), c) \quad (1)$$

where $c \in \mathcal{L}^n$ is an optional context that may include additional information or constraints. In this formulation, the VLM takes the language goal l , the current observation $\psi(s)$, and optional context c , and outputs a reward signal.

In robotics, the success of this formulation relies on the ability to describe tasks and goal states with precise language. For example, in manipulation tasks (Fig. 3 (a)), goals such as “Put carrot in bowl” are clear and unambiguous, allowing VLMs to effectively measure progress by comparing state-goal relationships in their embedding space $\mathcal{V} \subseteq \mathbb{R}^k$. In contrast, autonomous driving poses unique challenges, as the goal of “Safe driving” encompasses a wide range of acceptable behaviors and states. This abstract objective makes it difficult to establish clear semantic comparisons between the current vehicle state and the goal. While LORD (Ye et al., 2024) addresses this by using opposite language goals (Fig.3 (b)), this approach offers limited guidance by focusing only on states to avoid.

Drawing inspiration from human learning, where people often learn more effectively through contrasting goals, as exemplified by the steak-cooking scenario mentioned earlier, we propose using VLMs to generate semantic reward

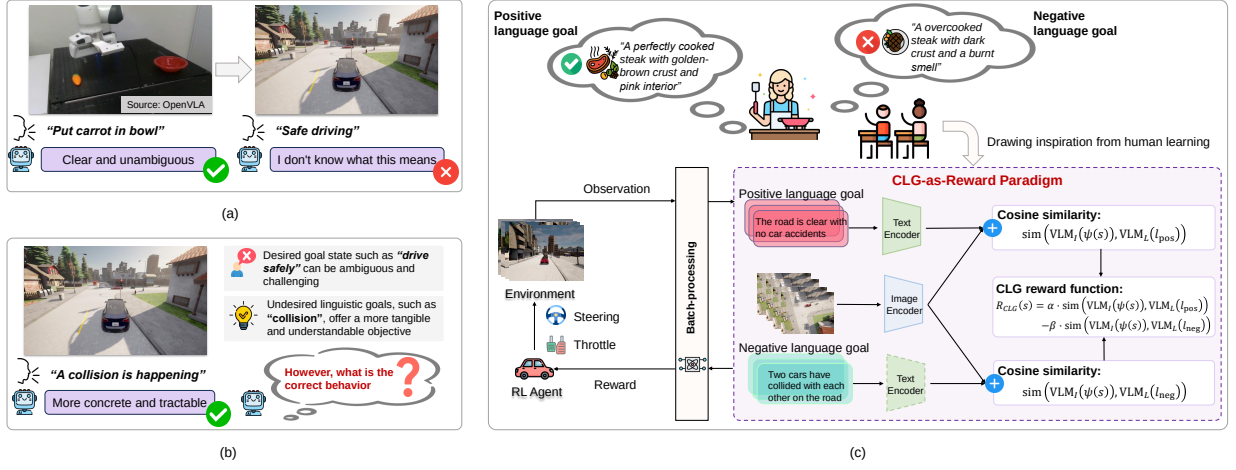


Figure 3: Conceptual comparisons of reward design paradigms. (a) Robotic manipulation tasks often feature well-defined goals (e.g., "Put carrot in bowl"), enabling VLMs to provide clear semantic rewards. (b) Existing methods that use only negative goals (e.g., "two cars have collided") focus on avoidance but lack positive guidance. (c) Our CLG-as-Reward paradigm integrates both positive and negative goals, allowing VLM-RL to deliver more informative semantic guidance for safer, more generalizable driving.

signals by aligning driving states with contrasting language descriptions (Fig.3 (c)). Specifically, we introduce the concept of **CLG**, which is defined as pairs of **positive and negative descriptions** that **encapsulate desired and undesired driving behaviors**.

Definition 1 (Contrasting Language Goal). Given a driving task $\mathcal{T}$, we define contrasting language goal as a pair $(l_{pos}, l_{neg}) \in \mathcal{L}^{\leq n} \times \mathcal{L}^{\leq n}$, where $l_{pos}, l_{neg} \in \mathcal{L}^{\leq n}$ denote positive and negative language goals respectively. Positive goals describe desired outcomes (e.g., "the road is clear with no car accidents"), while negative goals specify undesired scenarios (e.g., "two cars have collided with each other on the road"). Based on Eq. (1), the reward function of CLG is defined as

$$R_{CLG}(s) = VLM(l_{pos}, l_{neg}, \psi(s), c) \quad (2)$$

Specifically, we hope that the **positive component will guide the agent toward a desirable state while the negative component will prevent the agent from entering an undesirable state**. The ultimate goal is to **provide more informative reward signals** by encouraging desirable behaviors and punishing undesirable behaviors. Section 4.3.2 provides a detailed implementation of this idea.

4.3. CLG-based Semantic Reward Computation

4.3.1. VLM as Rewards **Revisited**

Safe driving tasks typically rely on sparse reward signals. In this setting, at each timestep t , given an observation $\psi(s_t)$ derived from state s_t , the agent executes an action $a_t \sim \pi_\theta(a_t|s_t)$ according to its policy π_θ . The environment then provides a sparse task reward r_t^{task} , typically defined as $r_t^{\text{task}} = \delta_{\text{success}}$, meaning a reward of 1 is received only upon task success and otherwise the reward is 0 (Cao et al., 2022). Such sparse rewards present substantial challenges for RL training, as they provide limited learning signals across the majority of the state space. A common approach is to manually design dense reward signals by combining metrics such as speed and distance to waypoints, either through simple summation (Wang et al., 2023a) or weighted aggregation (Chen et al., 2022). It is time-consuming, demands expertise, and may lead to conflicting sub-goals, resulting in suboptimal policies.

Building upon the general VLM reward formulation introduced in Eq. (1), recent works (Rocamonde et al., 2024; Baumli et al., 2023; Fu et al., 2024) have proposed augmenting these sparse task rewards with VLM-generated rewards r_t^{VLM} . This hybrid reward formulation can be expressed as

$$r_t = r_t^{\text{task}} + \rho \cdot r_t^{\text{VLM}} \quad (3)$$

where $\rho > 0$ is a weighting parameter that **balances the relative importance** of the VLM-generated reward against the sparse task reward.

Definition 2 (VLM-as-Reward Paradigm). Given the vision encoder $VLM_I : \mathcal{O} \rightarrow \mathcal{V}$ and language encoder $VLM_L : \mathcal{L}^{\leq n} \rightarrow \mathcal{V}$ that map into the same latent space $\mathcal{V} \subseteq \mathbb{R}^k$, and a sequence of state-action transitions $\{s_t, a_t, r_t, s_{t+1}\}_{t=1}^T$, the VLM reward is defined as

$$r_t^{\text{VLM}} = D(VLM_I(\psi(s_t)), VLM_L(l)) \quad (4)$$

where $D : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{R}$ is a distance metric between the embedded representations. Most works adopt CLIP (Radford et al., 2021) as the pre-trained VLM, employing cosine similarity as the distance metric (Rocamonde et al., 2024; Sontakke et al., 2024)

$$r_t^{\text{VLM}} = \text{sim}(\text{VLM}_I(\psi(s_t)), \text{VLM}_L(l)) = \frac{\text{VLM}_I(\psi(s_t))^\top \text{VLM}_L(l)}{\|\text{VLM}_I(\psi(s_t))\| \cdot \|\text{VLM}_L(l)\|} \quad (5)$$

where $\text{sim}(\cdot, \cdot)$ denotes the cosine similarity between embeddings, and VLM_I and VLM_L are the vision and language encoders, respectively. In this case, we don't need the context c . Here, the language goals l typically express desired behaviors. The computation process involves three steps: (a) The observation $\psi(s)$ is processed through the VLM_I to obtain a state embedding in the shared latent space $\mathcal{V}$. (b) The language goals are encoded via VLM_L to obtain their respective embeddings in the same space. (c) The reward is computed as the cosine similarity between the state embedding and the goal embedding.

While this formulation works well for robotics tasks with concrete goals, autonomous driving lacks such well-defined objectives. Recent work LORD (Ye et al., 2024) proposes using opposite language goals for safe driving tasks. Their key insight is that undesired states (e.g., collisions) are often more concrete and easier to specify than desired states. By defining the reward function $R_{\text{LORD}}(s)$ as

$$R_{\text{LORD}}(s) = 1 - \text{sim}(\text{VLM}_I(\psi(s)), \text{VLM}_L(l_{\text{neg}})) \quad (6)$$

The formulation aims to minimize the similarity between the current observation and the negative goal, thereby penalizing undesirable behaviors. By focusing solely on avoiding negative states, this approach may lack positive guidance for desirable behaviors, potentially limiting learning efficiency.

4.3.2. CLG as Reward Paradigm

Following the VLM-as-Reward paradigm in Definition 2, we propose a novel CLG-as-Reward paradigm specifically designed for safe driving tasks.

Definition 3 (CLG-as-Reward Paradigm). Given the CLG $(l_{\text{pos}}, l_{\text{neg}})$ introduced in Definition 1, we define the CLG reward function as

$$R_{\text{CLG}}(s) = \alpha \cdot \text{sim}(\text{VLM}_I(\psi(s)), \text{VLM}_L(l_{\text{pos}})) - \beta \cdot \text{sim}(\text{VLM}_I(\psi(s)), \text{VLM}_L(l_{\text{neg}})) \quad (7)$$

where $\alpha, \beta > 0$ are weighting factors satisfying $\alpha + \beta = 1$. If $\alpha > \beta$, the agent focuses more on achieving the positive goal, while if $\alpha < \beta$, the agent emphasizes steering clear of negative outcomes. For simplicity, in this work, we set $\alpha = \beta = 0.5$, i.e., the two goals are equally prioritized. $\text{sim}(\cdot, \cdot)$ denotes the cosine similarity between embeddings as defined in Eq. (5).

This formulation ensures that the agent is guided by both the positive and negative goals simultaneously. In other words, it encourages the agent to seek states similar to the positive goal while avoiding states similar to the negative goal, offering more informative guidance for policy learning. The following Thm. 1 formally establishes the effectiveness of the CLG-as-Reward paradigm.

Theorem 1 (Effectiveness of CLG-as-Reward Paradigm). Assume the VLM embeddings accurately capture the semantic content of observations and language goals. Under this assumption, optimizing the policy π to maximize the CLG reward R_{CLG} defined in Eq. (7) encourages the agent to simultaneously increase similarity to the positive goal and decrease similarity to the negative goal. As a result, the learned policy not only achieves the desired driving behaviors but also avoids undesirable ones.

Proof 1. The agent aims to maximize the expected discounted return:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^T \gamma^t R_{\text{CLG}}(s_t) \right] \quad (8)$$

An increase in $\text{sim}(\text{VLM}_I(\psi(s_t)), \text{VLM}_L(l_{\text{pos}}))$ indicates that the current state s_t more closely aligns with the positive goal. Since R_{CLG} adds a term proportional to this similarity, states that resemble the positive goal yield higher rewards. Conversely, R_{CLG} subtracts a term proportional to $\text{sim}(\text{VLM}_I(\psi(s_t)), \text{VLM}_L(l_{\text{neg}}))$, meaning states similar to the negative goal reduce the reward. Thus, maximizing R_{CLG} naturally pushes the agent toward states that are semantically closer to the positive goal and farther from the negative goal.

Formally, let s_t and s'_t be two potential subsequent states with embeddings $\mathbf{v}_t = \text{VLM}_I(\psi(s_t))$ and $\mathbf{v}'_t = \text{VLM}_I(\psi(s'_t))$. Define $\mathbf{v}_{\text{pos}} = \text{VLM}_L(l_{\text{pos}})$ and $\mathbf{v}_{\text{neg}} = \text{VLM}_L(l_{\text{neg}})$. If

$$\text{sim}(\mathbf{v}_t, \mathbf{v}_{\text{pos}}) - \text{sim}(\mathbf{v}_t, \mathbf{v}_{\text{neg}}) > \text{sim}(\mathbf{v}'_t, \mathbf{v}_{\text{pos}}) - \text{sim}(\mathbf{v}'_t, \mathbf{v}_{\text{neg}}) \quad (9)$$

then $R_{\text{CLG}}(s_t) > R_{\text{CLG}}(s'_t)$. The agent, through repeated interaction and policy updates, will tend to choose actions leading to s_t rather than s'_t , as s_t yields higher expected returns.

Over time, this consistent preference ensures the learned policy converges toward behaviors that improve positive similarity while reducing negative similarity. **Incorporating both positive and negative goals thus provides a more informative learning signal than using either one alone, resulting in superior policy learning.** Furthermore, as demonstrated theoretically in Appendix A, the CLG-as-Reward paradigm enhances the robustness of the learned policy, making it more resilient to uncertainty and adversarial perturbations.

4.4. Hierarchical Reward Synthesis

In this work, we follow the standard VLM-as-Reward paradigm, i.e., using only a language description of the task (Rocamonde et al., 2024; Sontakke et al., 2024; Wang et al., 2024). Yet, as noted by Fu et al. (2024), while zero-shot VLMs are effective in capturing coarse semantics, they often fall short in accurately representing fine-grained details. Furthermore, a single language description cannot comprehensively capture all the nuances of desired driving behaviors. As a result, relying solely on semantic rewards R_{CLG} could potentially mislead policy optimization in complex driving scenarios. Previous work has explored various strategies to address this issue: LAMP (Adeniji et al., 2023) uses **VLM-based reward for behavior pre-training**, ZSRM (Mahmoudieh et al., 2022) retrains VLMs with task-specific datasets, and FuRL (Fu et al., 2024) fine-tunes VLM representations and uses **relay RL technique**.

In contrast to these approaches, we aim to preserve the zero-shot capability of VLMs by integrating vehicle state information, which is readily available from on-board sensors, to generate **more stable and comprehensive reward signals**. In detail, We propose a hierarchical reward synthesis approach consisting of two phases: (a) **generating normalized semantic rewards** from VLMs and (b) combining **these semantic rewards** with **vehicle state information** to produce the synthesis reward signal.

Phase I: Semantic Reward Normalization. First, we compute the semantic rewards r_t^{CLG} by processing batches of observation frames through the CLIP. To ensure stability, we normalize the similarity scores to the range $[0, 1]$:

$$r_t'^{\text{CLG}} = \frac{\text{clip}(r_t^{\text{CLG}}, \theta_{\min}, \theta_{\max}) - \theta_{\min}}{\theta_{\max} - \theta_{\min}} \quad (10)$$

where $\theta_{\min}$ and $\theta_{\max}$ are empirically set to -0.03 and 0.0, respectively, to avoid extreme values and ensure consistent scaling. $\text{clip}(x, a, b)$ constrains x within the interval $[a, b]$.

Phase II: Integrating Vehicle State Information. We incorporate vehicle state information to produce the synthesis reward signal. This step leverages on-board sensor data to ensure the reward captures realistic driving behavior and safety constraints.

Definition 4 (Synthesis Reward Function). *The synthesis reward function $R_{\text{synthesis}} : \mathcal{S} \rightarrow \mathbb{R}$ is computed by combining the normalized semantic reward $r_t'^{\text{CLG}}$ with vehicle state information. Specifically:*

$$R_{\text{synthesis}}(s) = r_{\text{speed}}(s) \cdot f_{\text{center}}(s) \cdot f_{\text{angle}}(s) \cdot f_{\text{stability}}(s) \quad (11)$$

where r_{speed} modulates speed alignment, computed as $r_{\text{speed}} = 1 - \frac{|v - v_{\text{target}}|}{v_{\text{max}}}$ with $v_{\text{target}} = r_t'^{\text{CLG}} \cdot v_{\text{max}}$. $f_{\text{center}}(s)$ evaluates **the vehicle's lateral position relative to the lane center**. $f_{\text{angle}}(s)$ measures the **vehicle's orientation with respect to the road direction**. $f_{\text{stability}}(s)$ accounts for the **consistency of the vehicle's lateral position relative to the lane center**. Each term is bounded within $[0, 1]$.

Compared to traditional weighted-sum reward designs (Chen et al., 2022; Wang et al., 2023a), this multiplicative formulation naturally captures the interdependence of safety criteria without extensive parameter tuning. It yields an interpretable, stable, and easily implementable reward structure that leverages both semantic guidance from the VLM and actionable, high-fidelity vehicle state signals. The workflow of the hierarchical reward synthesis is shown in Appendix B as pseudocode. We also demonstrate the convergence and stability of the synthesis reward function in Appendix C and Appendix D.

Now, by combining the synthesis reward function in Eq. (11) with Eq. (3), we obtain the final reward function for the VLM-RL framework:

$$r_t' = r_t^{\text{task}} + \rho \cdot r_t^{\text{synthesis}} \quad (12)$$

This formulation allows the agent to benefit from both explicit task success signals and dense, context-aware rewards. The sparse task reward r_t^{task} ensures that the agent remains goal-oriented, while the synthesis reward $r_t^{\text{synthesis}}$ provides continuous feedback based on both high-level semantic understanding and low-level vehicle dynamics.

4.5. Policy Training with Batch-Processing

We adopt the soft actor-critic (SAC) algorithm (Haarnoja et al., 2018) as the backbone RL framework, due to its superior sample efficiency and stability in continuous control tasks. The SAC algorithm aims to maximize the expected return while encouraging exploration through entropy regularization. The objective can be written as:

$$J(\pi_\phi) = \mathbb{E}_{\pi_\phi} \left[\sum_{t=0}^T \gamma^t (R(s_t, a_t) + \alpha \mathcal{H}(\pi_\phi(\cdot | s_t))) \right] \quad (13)$$

where $\gamma \in [0, 1)$ is the discount factor, $\alpha > 0$ is the entropy temperature parameter controlling the trade-off between return and entropy maximization, and $\mathcal{H}(\pi_\phi(\cdot|s_t))$ is the entropy of the policy at state s_t .

To update the policy parameters ϕ , SAC minimizes the following objective:

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} [\mathbb{E}_{a_t \sim \pi_\phi(\cdot|s_t)} (-Q_\theta(s_t, a_t) + \alpha \log \pi_\phi(a_t|s_t))] \quad (14)$$

where $\mathcal{D}$ is the replay buffer, and Q_θ is the Q-function parameterized by θ .

The Q-function parameters θ are updated by minimizing the soft Bellman residual:

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - (r_t + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\phi(\cdot|s_{t+1})} [Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1}|s_{t+1})]) \right)^2 \right] \quad (15)$$

where $Q_{\bar{\theta}}$ is a target Q-function with periodically updated parameters $\bar{\theta}$.

Here, we replace the standard reward r_t in the soft Bellman residual with r'_t defined in Eq. (12):

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - (r'_t + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\phi(\cdot|s_{t+1})} [Q_{\bar{\theta}}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1}|s_{t+1})]) \right)^2 \right] \quad (16)$$

During training, the critic networks learn to estimate future returns based on Eq. (16), while the policy network learns to maximize these returns through the standard SAC policy gradient updates.

To address the computational bottleneck of CLIP inference, we develop a batch-processing technique. During environment interaction, **tuples of $(o_t, s_t, a_t, r_t, o_{t+1}, s_{t+1})$ are stored in a replay buffer.** Here, o_t represents the raw observation image required for CLIP processing, and s_t contains the processed state information for policy learning. At predefined intervals, we sample a batch of observations from the replay buffer and process them through the CLIP encoder. The CLIP embeddings of the CLG (l_{pos} and l_{neg}) are computed only once at the start of training, as they remain constant. We compute the synthesized rewards according to Eq.(11), which then are used to **update the stored transitions in the replay buffer.** The SAC algorithm subsequently samples these updated transitions via its standard update procedure for policy optimization. This approach effectively **decouples the computationally expensive reward computation from the main RL training loop,** enabling the agent to continue learning while rewards are computed asynchronously. The complete training procedure is outlined in Appendix E.

5. Experiments and Results

5.1. Experiment Setting

5.1.1. RL Setups

The RL agent takes three types of inputs: (1) a bird’s-eye view (BEV) semantic segmentation image that captures the surrounding environment, including drivable areas, lane boundaries, and other traffic participants, as illustrated in Fig. 4 (c)-(e). This provides crucial spatial information for navigation and obstacle avoidance. (2) ego state information consisting of the current steering angle, throttle value, and vehicle speed. These values reflect the vehicle’s dynamic state and are essential for maintaining smooth control. (3) future navigation information represented by the next 15 waypoints along the planned route. Each waypoint is defined by its (x, y) coordinates relative to the vehicle’s current position, helping the agent understand and follow the desired trajectory.

The action space is designed as a continuous 2-dimensional space $[-1, 1]^2$, where each dimension controls different aspects of vehicle motion. The first dimension corresponds to the steering angle, with values in $[-1, 1]$ representing the full range of steering control. Specifically, -1 indicates maximum left turn, 0 represents straight ahead, and $+1$ indicates maximum right turn. The second dimension combines throttle and brake control in a single value range $[-1, 1]$. When this value is positive ($[0, 1]$), it directly maps to the throttle intensity, with 1 representing full throttle. Conversely, when the value is negative, its absolute value maps to brake intensity, where -1 corresponds to full brake. An episode is terminated when any of the following conditions are met: (a) collision with any obstacles, vehicles, or pedestrians, (b) deviation from the road center line by more than 3 meters, or (c) vehicle speed remains below 1 km/h for more than 90 consecutive seconds, indicating the agent is stuck or unable to progress. These termination conditions are designed to enforce safe driving behavior and ensure efficient navigation progress.

We build our implementation upon the Stable-Baselines3 library (Raffin et al., 2021), which provides reliable and well-tested implementations of modern RL algorithms. Stable-Baselines3 offers a modular design and stable performance, allowing us to focus on extending the core algorithms rather than implementing them from scratch. Specifically, we extend the standard implementations of SAC and PPO to incorporate our CLG-based and hierarchical reward computation during the training process. The policy network architecture is specifically designed for processing heterogeneous input types: we employ a 6-layer CNN to extract features from the BEV semantic segmentation images, while using MLPs to process both the ego state information and future navigation waypoints. These processed features are then concatenated before being fed into the final policy head for action prediction.

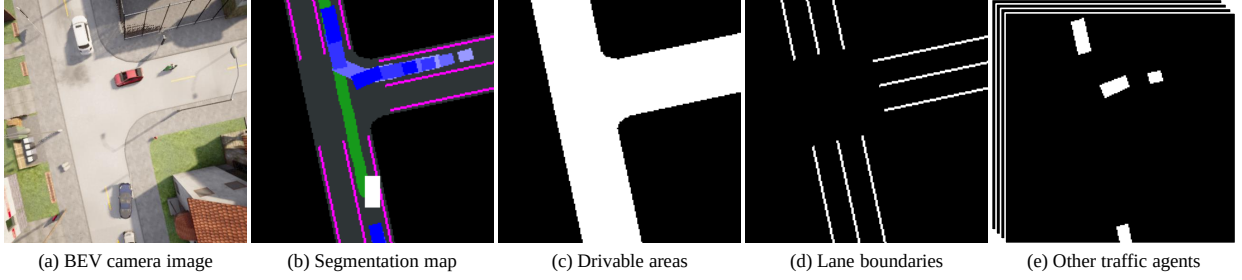


Figure 4: Bird's eye view of RL agent's surrounding environment, where the purple vehicle in (a) and the white box in (b) represents the RL agent.

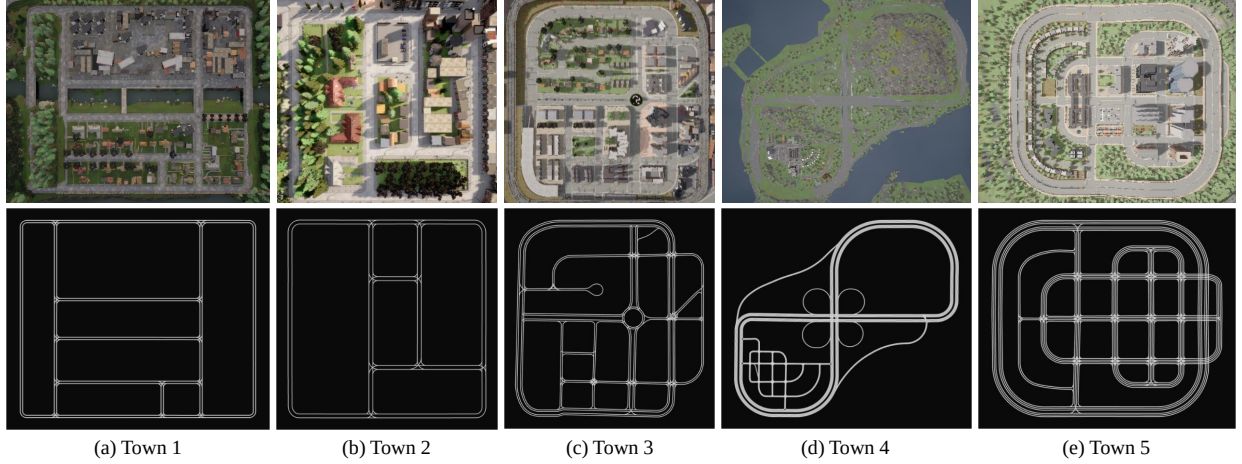


Figure 5: Bird's eye view of Towns and their drivable routes in CARLA.

5.1.2. Driving Scenarios

We train all models in CARLA's Town 2 map to ensure a fair comparison and evaluate the effectiveness of our approach and all baseline models. As shown in Fig. 5 (b), this town presents a typical European-style urban layout with a variety of challenging driving scenarios. It consists of several interconnected areas including a residential district, a commercial zone with single-lane roads, and complex intersections controlled by traffic lights. The compact nature of Town 2 makes it particularly suitable for evaluation, as it provides diverse driving conditions within a manageable scale, including both straight roads and curved segments, multiple T-junctions, and different types of lane markings and road geometries. These features create challenging scenarios for assessing both basic driving capabilities and complex decision-making behaviors. In Section 5.7.1, we further evaluate the generalization ability of our approach in Towns 1, 3, 4, and 5, as shown in Figs. 5 (a), (c), (d), and (e), respectively. Unless otherwise specified, all results are reported based on experiments conducted in Town 2.

To create a more realistic and challenging environment, we populate the town with 20 vehicles running in autopilot mode. These vehicles are randomly spawned across the map and operate using CARLA's built-in traffic manager, which enables them to follow traffic rules, respond to traffic lights, and perform basic collision avoidance. This dynamic traffic flow significantly increases the complexity of the learning task for our RL agent, as it must now handle various interactive scenarios such as car following, overtaking, and yielding to other vehicles. The presence of multiple moving vehicles not only makes the environment more similar to real-world urban driving conditions but also challenges the RL agent to develop more robust and adaptive driving strategies.

5.1.3. Navigation Routes

We dynamically assign navigation routes to the RL agent during training and evaluation. At each reset, we utilize the 101 predefined spawn points available on the drivable routes in Fig. 5 (b) as potential starting and destination locations. Specifically, we randomly select two distinct spawn points to serve as the start and end points, then employ the A* search algorithm to compute the shortest path between them, which becomes the navigation route for the agent. Notably, instead of terminating the episode upon reaching the destination, we continuously generate new navigation routes for the agent by repeating this random selection and path planning process. This dynamic route assignment continues until the cumulative driving distance within the episode reaches 3000 meters, allowing us to evaluate the agent's performance across diverse navigation scenarios in a single episode.

5.1.4. CLIP Config

We employ the CLIP model (Radford et al., 2021) as our foundational VLM for CLG-based semantic reward generation. Specifically, we utilize OpenCLIP's ViT-bigG-14 model pre-trained on the LAION-2B dataset with 2.32

English billion image-text pairs (Schuhmann et al., 2022). The model applies a patch size of 14×14 pixels and accepts images with a resolution of 224×224 pixels as input, which we obtain by resizing the original CARLA camera images. During inference, we leverage CLIP’s visual encoder to extract high-dimensional feature representations from the driving scenes, while the text encoder processes our predefined CLG. All CLIP components are kept frozen during our experiments to maintain stable and consistent semantic reward generation.

5.2. Evaluation Metrics

To comprehensively evaluate the performance and safety aspects of our autonomous driving system, we employ multiple quantitative metrics that assess both driving efficiency and safety characteristics. For driving efficiency assessment, we measure the average speed (**AS**) maintained by the vehicle throughout episodes, the route completion (**RC**) which represents the number of successfully completed routes during one episode, and the total traveled distance (**TD**) which captures the cumulative distance covered by the vehicle during each episode.

Safety performance is evaluated through several complementary metrics. The fundamental collision rate (**CR**) measures the percentage of episodes containing collision events. We further analyze collision patterns through two frequency metrics: time-based collision frequency (**TCF**), measuring collisions per 1000 time steps, and distance-based collision frequency (**DCF**), measuring collisions per kilometer traveled. To assess collision severity, we record the collision speed (**CS**) at the moment of each collision. Additionally, we track the inter-collision time steps (**ICT**), which measure the average number of time steps between consecutive collision events, providing insights into the temporal distribution of safety incidents. In the test phase, we also report the success rate (**SR**) to evaluate the model’s ability to successfully reach the destination across 10 predefined routes.

5.3. Baselines

We compare our method against state-of-the-art baselines, which can be categorized into two primary groups: expert-designed reward methods and LM-designed reward methods.

Expert-designed Reward Methods. We implement the following baselines with manually designed reward functions using both SAC and PPO. These methods include binary rewards that only consider collision states, and summation rewards that combine multiple weighted terms to guide driving behavior:

- **TIRL** (Cao et al., 2022) employs a simple binary reward that only penalizes collision states with -1 and assigns 0 reward to all other states.
- **Chen-SAC** (Chen et al., 2022) employs a reward function that penalizes collisions, speeding, running out of lane, and excessive steering, while incentivizing forward velocity and controlled lateral acceleration to guide autonomous driving decisions.
- **ASAP-RL-PPO** (Wang et al., 2023a) employs a reward function that provides positive incentives for forward progress, reaching the destination, and overtaking vehicles, while applying penalties for collisions with other vehicles or road curbs.
- **ChatScene** (Zhang et al., 2024) designs a weighted sum of reward terms that encourages smooth driving behaviors (longitudinal speed, lateral acceleration, and steering control) while penalizing unsafe actions (collisions, out-of-lane driving, and speeding). A small constant reward is added as baseline incentive to facilitate learning.

LM-designed Reward Methods. We also compare against recent approaches that leverage language models for reward design, including both LLM-based and VLM-based methods:

- **Revolve** and **Revolve-auto** (Hazra et al., 2024) introduce an evolutionary framework that utilizes LLMs to generate reward function code based on human feedback. For simplicity, we adopt their best-performing reward function provided in the paper for comparison.
- **VLM-SR** (Baumli et al., 2023) uses off-the-shelf VLMs like CLIP to generate binary reward signals by calculating the cosine similarity between image observations and language goals, followed by softmax normalization and thresholding to determine goal achievement.
- **RoboCLIP** (Sontakke et al., 2024) generates sparse reward signals at the end of each episode by computing the similarity between video observations of agent trajectories and a task descriptor. Due to the complexity of autonomous driving, we adapt this baseline by using CLIP to compute the similarity between each frame’s image observation and the language goal, generating a dense reward signal.
- **VLM-RM** (Rocamonde et al., 2024) uses a baseline-target approach to project the current state embedding onto the direction between a baseline state (e.g., “a car”) and a target state (e.g., “a car is driving safely”).
- **LORD** (Ye et al., 2024) focuses on negative reward generation through VLMs to penalize unsafe driving behaviors, using concrete undesired states like “collision” to shape the reward signal. We also adapt this baseline by incorporating a dense speed-based reward, denoted as **LORD-Speed**, to further guide the agent’s behavior based on driving speed.

Table 1: Performance comparison with baselines during training. Mean and standard deviation over 3 seeds. The best results are marked in **bold**.

Model	Reference	AS $\uparrow$	RC $\uparrow$	TD $\uparrow$	CS $\downarrow$	CR $\downarrow$	ICT $\uparrow$	DCF $\downarrow$	TCF $\downarrow$
<i>Expert-designed Reward Methods (Binary Rewards)</i>									
TIRL-SAC	TR-C'22	0.01 $\pm$ 0.01	0.01 $\pm$ 0.003	0.21 $\pm$ 0.13	3.0 $\pm$ 1.0	0.013 $\pm$ 0.01	54965 $\pm$ 15734	14009 $\pm$ 3864	14.0 $\pm$ 0.90
TIRL-PPO	TR-C'22	0.26 $\pm$ 0.25	0.04 $\pm$ 0.23	2.95 $\pm$ 1.46	2.9 $\pm$ 0.8	0.12 $\pm$ 0.07	32507 $\pm$ 16076	3290 $\pm$ 2368	5.50 $\pm$ 2.91
<i>Expert-designed Reward Methods (Summation Rewards)</i>									
Chen-SAC	T-ITS'22	19.9 $\pm$ 1.07	0.68 $\pm$ 0.12	147.9 $\pm$ 32.2	9.14 $\pm$ 3.66	0.12 $\pm$ 0.04	2325 $\pm$ 518	20.3 $\pm$ 4.8	6.2 $\pm$ 0.94
ASAP-RL-PPO	RSS'23	2.86 $\pm$ 1.11	0.04 $\pm$ 0.002	3.99 $\pm$ 0.37	2.25 $\pm$ 0.07	0.21 $\pm$ 0.13	9276 $\pm$ 4064.5	394.4 $\pm$ 201.2	26.1 $\pm$ 19.5
ChatScene-SAC	CVPR'24	17.4 $\pm$ 0.20	2.0 $\pm$ 0.24	717 $\pm$ 68	10.1 $\pm$ 0.6	0.88 $\pm$ 0.02	1774 $\pm$ 124	4.6 $\pm$ 0.5	1.63 $\pm$ 0.13
ChatScene-PPO	CVPR'24	14.1 $\pm$ 0.14	0.9 $\pm$ 0.13	248 $\pm$ 44	5.2 $\pm$ 0.1	0.83 $\pm$ 0.02	793 $\pm$ 69	14.1 $\pm$ 0.4	3.62 $\pm$ 0.02
<i>LLM-based Reward Methods</i>									
Revolve	ICLR'25	17.6 $\pm$ 0.71	1.9 $\pm$ 1.07	671 $\pm$ 437	9.5 $\pm$ 2.9	0.76 $\pm$ 0.28	1556 $\pm$ 875	20.5 $\pm$ 25.1	5.48 $\pm$ 6.19
Revolve-auto	ICLR'25	17.3 $\pm$ 0.34	1.4 $\pm$ 0.07	485 $\pm$ 11	6.0 $\pm$ 2.4	0.83 $\pm$ 0.23	1390 $\pm$ 403	6.7 $\pm$ 1.4	2.27 $\pm$ 0.36
<i>VLM-based Reward Methods</i>									
VLM-SR	NeurIPS'23	0.31 $\pm$ 0.17	0.05 $\pm$ 0.01	5.60 $\pm$ 0.91	3.03 $\pm$ 0.36	0.19 $\pm$ 0.07	15096 $\pm$ 4551.5	420.0 $\pm$ 385.0	3.89 $\pm$ 1.16
RoboCLIP	NeurIPS'23	0.47 $\pm$ 0.23	0.22 $\pm$ 0.11	34.0 $\pm$ 21.2	4.04 $\pm$ 0.59	0.29 $\pm$ 0.05	12598 $\pm$ 3002	85.3 $\pm$ 46.1	2.00 $\pm$ 0.54
VLM-RM	ICLR'24	0.15 $\pm$ 0.09	0.07 $\pm$ 0.05	7.51 $\pm$ 5.76	3.0 $\pm$ 0.5	0.16 $\pm$ 0.08	17511 $\pm$ 923	918 $\pm$ 77.6	3.53 $\pm$ 2.11
LORD	Arxiv'24	0.08 $\pm$ 0.06	0.04 $\pm$ 0.02	4.94 $\pm$ 3.81	3.6 $\pm$ 1.4	0.095 $\pm$ 0.03	16852 $\pm$ 1826	2904 $\pm$ 1043	6.93 $\pm$ 2.70
LORD-Speed	Arxiv'24	18.1 $\pm$ 0.73	2.15 $\pm$ 0.82	816.9 $\pm$ 397.5	8.28 $\pm$ 2.47	0.92 $\pm$ 0.01	1954 $\pm$ 918	5.4 $\pm$ 3.8	1.93 $\pm$ 1.32
VLM-RL (ours)	-	17.4 $\pm$ 0.24	4.4 $\pm$ 0.25	1780 $\pm$ 139	2.6 $\pm$ 0.7	0.68 $\pm$ 0.03	5920 $\pm$ 725	2.5 $\pm$ 0.7	0.76 $\pm$ 0.14

5.4. Main Results

We present a detailed evaluation of our proposed VLM-RL against various baseline methods in Tabs. 1-2 and Figs. 6-9. All experiments were conducted using three different random seeds and trained for 1 million steps to ensure statistical significance. The performance metrics reported in Tab. 1 represent the mean and standard deviation of the final checkpoint during training across these three independent runs. For testing results, we selected the best-performing checkpoint from each training run based on comprehensive performance metrics, with the corresponding evaluation results presented in Tab. 2. The learning curves shown in Figs. 6-9 track the training progress of different methods, where solid lines indicate the mean performance across three seeds, and the shaded regions represent one standard deviation from the mean. This visualization allows us to observe not only the final performance but also the learning dynamics and stability of different approaches throughout the training process.

5.4.1. Training Performance Analysis

We first compare VLM-RL with expert-designed reward methods. From the training curves in Fig. 6, we can see that TIRL exhibits relatively low collision rates and high collision-free intervals. However, this seemingly positive performance is actually a result of the agent's failure to learn basic driving behaviors. As shown in Tab. 1, TIRL-SAC achieves only 0.01 km/h average speed, 0.01 route completion, and 0.21m total driving distance, indicating that the agent essentially remains stationary rather than learning to navigate. In contrast, our VLM-RL demonstrates superior performance across all key metrics. It achieves an average speed of 17.4 km/h while maintaining a low collision speed of 2.6 km/h, and most importantly, successfully completes 4.4 routes with a total driving distance of 1780m. This comprehensive performance indicates that VLM-RL successfully learns both safe driving behaviors and effective navigation strategies. The poor performance of TIRL can be attributed to the limitations of its simple binary reward design in the context of autonomous driving. Binary rewards that only penalize collisions (-1) while assigning neutral rewards (0) to all other states create a significant exploration challenge. In autonomous driving, where the action space is continuous and the state space is highly complex, such sparse binary rewards provide insufficient learning signals for the agent to discover productive driving behaviors. Without positive reinforcement for forward progress or successful navigation, the agent only learns to minimize collision risk by remaining stationary. This represents a local optimum that avoids negative rewards but fails to accomplish the actual driving objectives.

Compared to other expert-designed reward methods with weighted summation terms, VLM-RL demonstrates more balanced and stable performance. Chen-SAC achieves a higher average speed of 19.9 km/h but suffers from

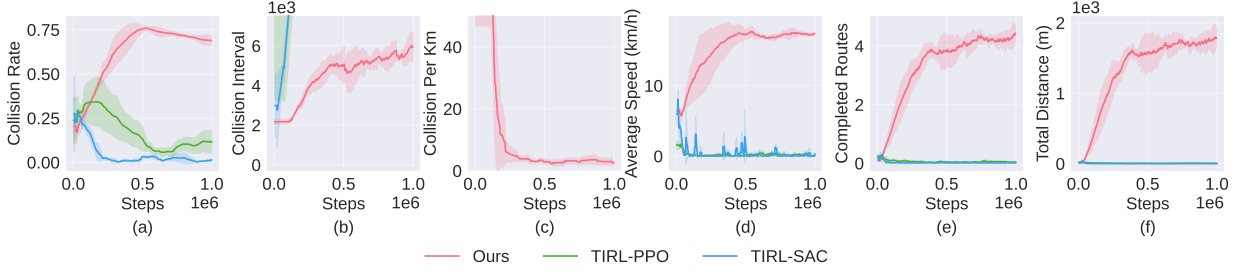


Figure 6: Performance comparison with expert-designed baselines with binary rewards during training.

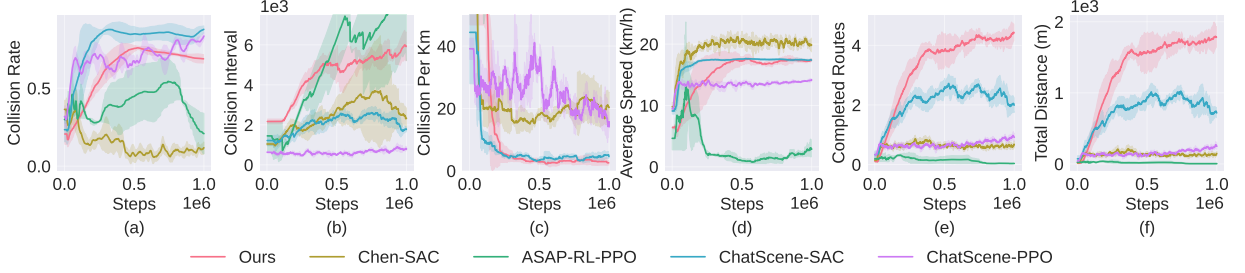


Figure 7: Performance comparison with expert-designed baselines with summation rewards during training.

a high collision frequency of DCF and TCF and low route completion of 0.68 routes, suggesting that its reward design over-emphasizes speed at the expense of safety. ASAP-RL-PPO maintains relatively safe driving with a low collision rate of 0.21 but achieves limited progress with only 0.04 routes. This indicates its reward function may be too conservative, leading to overly cautious driving behaviors. ChatScene variants show moderate performance across metrics but struggle to balance driving efficiency and safety, with their collision rates of 0.88 and 0.83 significantly higher than VLM-RL’s 0.68. As shown in Fig. 7, our method exhibits more stable and consistent learning progress. While other methods often show high variance or unstable behaviors during training, VLM-RL maintains steady improvement across all metrics. It is worth noting that the continuous decrease in collision rate during the latter half of training demonstrates the agent’s improving ability to avoid dangerous situations while maintaining high driving efficiency. This learning pattern is unique to our method, as other approaches either show fluctuating collision rates or achieve safety at the cost of progress. Additionally, our method achieves the highest route completion of 4.4 and total driving distance of 1780m among all methods. This superior performance demonstrates that our CLG-based rewards and hierarchical reward synthesis provide more informative and balanced learning signals compared to manually designed reward functions, effectively guiding the agent to learn both safe and efficient driving behaviors.

Next, we compare our method with LLM-based reward approaches. As shown in Tab. 1, both Revolve and Revolve-auto achieve comparable driving speeds (17.6 km/h and 17.3 km/h) but exhibit relatively high collision rates (0.76 and 0.83) and limited route completion numbers (1.9 and 1.4). The training curves in Fig. 8 reveal interesting behavioral patterns. While Revolve variants quickly learn to achieve and maintain high average speeds comparable to VLM-RL, they struggle with safety aspects, as evidenced by their consistently high collision rates throughout training. In contrast, VLM-RL demonstrates a more balanced learning trajectory, gradually improving both driving efficiency and safety. Most notably, while Revolve and Revolve-auto show limited improvement in route completion and total distance traveled after the initial learning phase, VLM-RL continues to make steady progress, ultimately achieving more than twice the route completion rate (4.4 versus 1.9) and significantly longer driving distances (1780m versus 671m). This suggests that our CLG-based hierarchical reward design provides more comprehensive and well-structured learning signals compared to LLM-generated reward functions, enabling the agent to better balance the competing objectives of efficiency and safety in autonomous driving.

When comparing with VLM-based reward methods, we observe distinct performance patterns. VLM-SR, RoboCLIP, and VLM-RM, despite their demonstrated success in robotic tasks, show limited effectiveness in autonomous driving scenarios when relying solely on VLM-derived semantic rewards. These methods achieve very low average speeds of 0.31, 0.47, and 0.15 km/h respectively, and route completion numbers all below 0.22, suggesting they struggle to learn basic driving behaviors. The training curves in Fig. 9 (a)-(f) reveal that these methods maintain relatively low collision rates primarily because the agents remain nearly stationary, similar to the behavior observed with TIRL. This performance gap between robotic manipulation and autonomous driving stems from fundamental task differences. Robotic tasks typically involve discrete, well-defined goal states such as grasping objects or manipulating tools that can be effectively captured by VLM-based similarity metrics. In contrast, autonomous driving requires continuous, dynamic decision-making where the desired behavior is a complex combination of multiple objectives that evolve over time. Our hierarchical reward synthesis approach addresses this challenge by combining high-level semantic un-

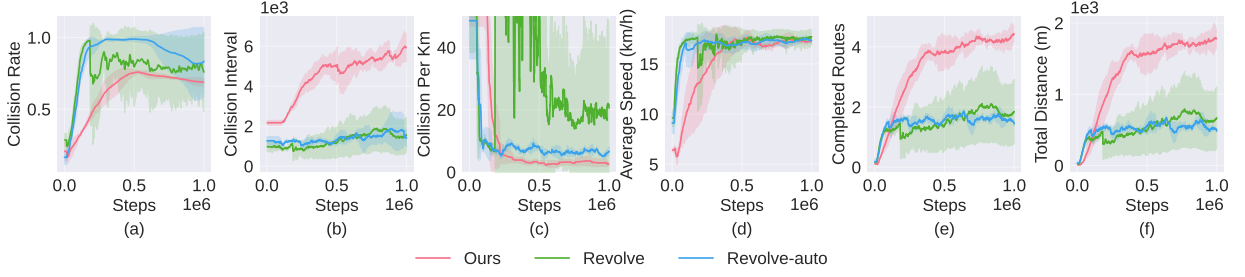


Figure 8: Performance comparison with LLM-designed baselines during training.

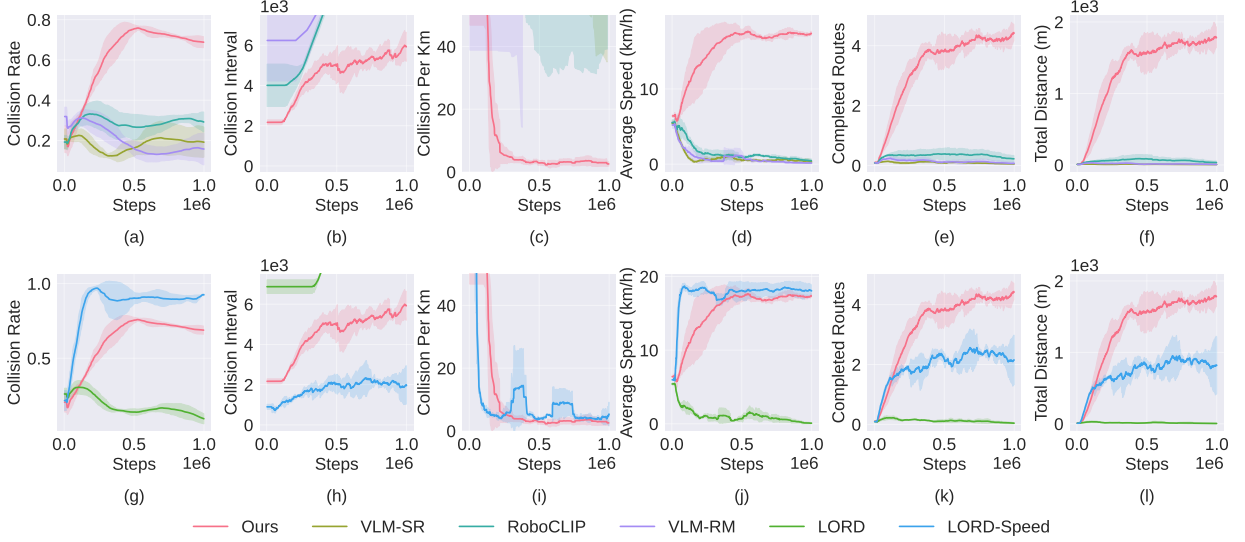


Figure 9: Performance comparison with VLM-designed baselines during training.

derstanding from CLG with low-level vehicle state information, providing comprehensive learning signals that better guide the driving policy.

LORD and its variant LORD-Speed, both designed specifically for autonomous driving, show contrasting performance patterns. While LORD achieves a low collision rate of 0.095, its average speed of 0.08 km/h and route completion of 0.04 indicate similar limitations as other VLM-only approaches. It is worth noting that LORD’s original success was demonstrated in the HighwayEnv simulator (Leurent, 2018) with discrete action spaces and simplified observation representation, which is considerably less complex than our CARLA environment with continuous control and realistic visual inputs. LORD-Speed, which incorporates additional speed-based rewards, shows significantly improved performance with an average speed of 18.1 km/h and a route completion rate of 2.15. However, this improvement comes at the cost of safety, as evidenced by its high collision rate of 0.92. As shown in Fig. 9 (g)-(l), LORD-Speed quickly achieves high average speeds but fails to effectively balance safety and efficiency, maintaining consistently high collision rates throughout training. In contrast, VLM-RL demonstrates more balanced learning progress. This superior performance validates the effectiveness of our CLG-based and hierarchical reward design in providing more comprehensive and balanced learning signals compared to existing VLM-based approaches.

5.4.2. Performance Evaluation in Testing

To further validate the effectiveness of VLM-RL, we conduct comprehensive testing evaluations across 10 predefined routes and compare the performance with baseline methods. The route completion metric represents the average route completion rates during each evaluation episode. The testing results in Tab. 2 demonstrate significant advantages of our approach compared to the baselines.

The limitations of binary reward methods remain evident in the testing phase. TIRL variants achieve a route completion rate of 0.01 and total driving distances of 4.7m and 14.8m respectively, confirming their failure to learn meaningful driving behaviors. Among expert-designed reward methods with weighted summation terms, Chen-SAC maintains the highest average speed at 21.4 km/h but shows limited effectiveness with a 0.08 success rate and 0.29 route completion, indicating its aggressive driving style compromises mission success. ChatScene variants demonstrate more balanced performance with success rates of 0.73 and 0.63 respectively, though their collision speeds of 1.18 km/h and 0.89 km/h suggest potential safety concerns.

LLM-based approaches demonstrate competitive performance during testing, with Revolve achieving a success rate of 0.83 and route completion of 0.92. However, their collision speeds of 1.53 km/h and 1.65 km/h indicate

Table 2: Performance comparison with baselines during testing. Mean and standard deviation over 3 seeds. The best results are marked in **bold**.

Model	Reference	AS $\uparrow$	RC $\uparrow$	TD $\uparrow$	CS $\downarrow$	SR $\uparrow$
<i>Expert-designed Reward Methods (Binary Rewards)</i>						
TIRL-SAC	TR-C'22	0.37 $\pm$ 0.28	0.01 $\pm$ 0.001	4.7 $\pm$ 3.5	0.24 $\pm$ 0.34	0.0 $\pm$ 0.0
TIRL-PPO	TR-C'22	0.43 $\pm$ 0.23	0.01 $\pm$ 0.005	14.8 $\pm$ 9.7	0.10 $\pm$ 0.15	0.0 $\pm$ 0.0
<i>Expert-designed Reward Methods (Summation Rewards)</i>						
Chen-SAC	T-ITS'22	21.4 $\pm$ 1.16	0.29 $\pm$ 0.12	663.6 $\pm$ 286.7	2.07 $\pm$ 2.21	0.08 $\pm$ 0.08
ASAP-RL-PPO	RSS'23	1.25 $\pm$ 0.30	0.01 $\pm$ 0.00	28.1 $\pm$ 3.44	0.61 $\pm$ 0.61	0.0 $\pm$ 0.0
ChatScene-SAC	CVPR'24	17.7 $\pm$ 0.12	0.88 $\pm$ 0.03	1763.2 $\pm$ 90.9	1.18 $\pm$ 0.46	0.73 $\pm$ 0.05
ChatScene-PPO	CVPR'24	15.3 $\pm$ 0.33	0.78 $\pm$ 0.05	1515.6 $\pm$ 129.1	0.89 $\pm$ 0.32	0.63 $\pm$ 0.05
<i>LLM-based Reward Methods</i>						
Revolve	ICLR'25	18.4 $\pm$ 0.03	0.92 $\pm$ 0.11	1915.3 $\pm$ 248.3	1.53 $\pm$ 2.16	0.83 $\pm$ 0.24
Revolve-auto	ICLR'25	17.2 $\pm$ 0.76	0.80 $\pm$ 0.06	1539.6 $\pm$ 147.5	1.65 $\pm$ 0.28	0.63 $\pm$ 0.05
<i>VLM-based Reward Methods</i>						
VLM-SR	NeurIPS'23	0.53 $\pm$ 0.27	0.02 $\pm$ 0.00	47.9 $\pm$ 9.2	0.18 $\pm$ 0.25	0.0 $\pm$ 0.0
RoboCLIP	NeurIPS'23	0.44 $\pm$ 0.05	0.07 $\pm$ 0.03	146.3 $\pm$ 62.3	1.05 $\pm$ 0.58	0.0 $\pm$ 0.0
VLM-RM	ICLR'24	0.20 $\pm$ 0.05	0.02 $\pm$ 0.01	35.9 $\pm$ 25.8	0.003 $\pm$ 0.005	0.0 $\pm$ 0.0
LORD	Arxiv'24	0.17 $\pm$ 0.08	0.02 $\pm$ 0.02	45.1 $\pm$ 57.1	0.02 $\pm$ 0.02	0.0 $\pm$ 0.0
LORD-Speed	Arxiv'24	18.9 $\pm$ 0.36	0.87 $\pm$ 0.08	1783.4 $\pm$ 172.8	2.80 $\pm$ 1.16	0.67 $\pm$ 0.05
VLM-RL (ours)	-	19.3 $\pm$ 1.29	0.97 $\pm$ 0.03	2028.2 $\pm$ 96.6	0.02 $\pm$ 0.03	0.93 $\pm$ 0.04

persistent safety issues. Most VLM-based methods, including VLM-SR, RoboCLIP, VLM-RM, and LORD, exhibit highly conservative behaviors with route completion rates below 0.07 and success rates of 0.0. LORD-Speed shows significantly improved efficiency metrics but records the highest collision speed at 2.80 km/h among all methods.

In contrast, VLM-RL achieves superior performance across all key metrics during testing. It maintains a high average speed of 19.3 km/h while recording a low collision speed of 0.02 km/h, matching the safety level of the most conservative approaches. Most notably, VLM-RL achieves the highest success rate of 0.93 and route completion of 0.97, along with the longest total driving distance of 2028.2m. These results demonstrate that our method not only learns more effective driving policies but also exhibits better generalization to testing scenarios. The significant improvements in both efficiency and safety metrics validate the effectiveness of our CLG-based and hierarchical reward design in providing comprehensive and well-balanced learning signals for safe driving tasks.

5.5. Ablation Study

Building upon our previous baseline comparisons with VLM-SR, RoboCLIP, VLM-RM and LORD, which established the advantages of our hierarchical reward synthesis approach, we conduct ablation studies to further validate the effectiveness of our proposed CLG approach. Specifically, we investigate the performance when using only positive language goals (VLM-RL-pos) and only negative language goals (VLM-RL-neg), respectively. These variants allow us to analyze the individual contribution of each goal type and demonstrate why combining both through our contrasting framework leads to superior performance. Additionally, we compare the performance of using CARLA's built-in segmentation camera-based BEV as the RL agent's observation (VLM-RL-bev) as an ablation experiment to validate the effectiveness of the BEV design shown in Fig. 4. These ablation experiments provide additional insights into the specific mechanisms that contribute to our method's effectiveness.

As shown in Fig. 10, we can observe clear patterns that demonstrate the advantages of combining both positive and negative language goals in our full VLM-RL model compared to its variants. In terms of safety metrics, the collision rate shows that both VLM-RL-pos and VLM-RL-neg tend to converge to a higher collision rate. The collision interval further supports this observation, with VLM-RL-full maintaining significantly longer intervals between collisions, reaching nearly 6000 steps compared to around 2500-3000 steps for the variants. Regarding driving efficiency, the average speed indicates that all three models eventually achieve comparable speeds of around 17-18 km/h. The most striking differences appear in the completed routes and total distance traveled, where VLM-RL-full significantly

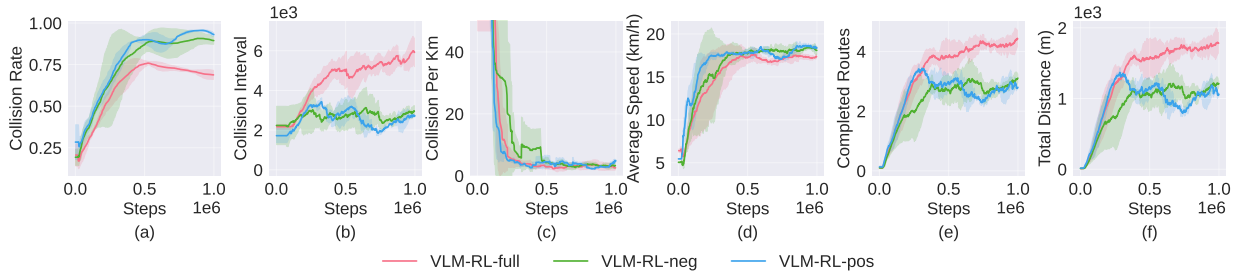


Figure 10: Performance comparison using different language goals during training.

Table 3: Ablation study on the model components during testing. Mean and standard deviation over 3 seeds. The best results are marked in **bold**.

Model	AS $\uparrow$	RC $\uparrow$	TD $\uparrow$	CS $\downarrow$	SR $\uparrow$
VLM-RL-bev	18.5 $\pm$ 1.58	0.92 $\pm$ 0.05	1905.5 $\pm$ 129.7	0.48 $\pm$ 0.51	0.78 $\pm$ 0.07
VLM-RL-neg	19.2 $\pm$ 0.72	0.90 $\pm$ 0.08	1901.5 $\pm$ 228.1	0.94 $\pm$ 0.95	0.77 $\pm$ 0.09
VLM-RL-pos	19.4 $\pm$ 0.88	0.89 $\pm$ 0.10	1817.3 $\pm$ 263.3	1.47 $\pm$ 1.34	0.75 $\pm$ 0.05
VLM-RL-full	19.3 $\pm$ 1.29	0.97 $\pm$ 0.03	2028.2 $\pm$ 96.6	0.02 $\pm$ 0.03	0.93 $\pm$ 0.04

outperforms both variants, completing 4.4 routes compared to about 3 routes for the variants, and covering nearly 1800m versus approximately 1200m.

The testing results in Tab. 3 further validate these observations and provide insights into the effectiveness of our BEV design. While VLM-RL-bev achieves competitive performance in terms of average speed (18.5 km/h) and route completion (0.92), its collision speed of 0.48 km/h and success rate of 0.78 indicate compromised safety compared to our full model. This suggests that our custom BEV design better captures critical environmental features for safe navigation. The single-goal variants (VLM-RL-neg and VLM-RL-pos) show similar patterns during testing, with high average speeds but elevated collision speeds of 0.94 km/h and 1.47 km/h respectively. In contrast, VLM-RL-full maintains comparable efficiency while achieving a remarkably low collision speed of 0.02 km/h and the highest success rate of 0.93. These results demonstrate that the CLG approach leads to more balanced and effective learning, enabling the agent to better navigate the trade-off between safety and efficiency in autonomous driving.

5.6. VLM-RL Performance Scaling Across CLIP Model Sizes

To systematically investigate how the scale of VLM affects the performance of VLM-RL, we conducted experiments with four different CLIP model variants of increasing size and complexity: ViT-B-32 (base), ViT-L-14-quickgelu (large), ViT-H-14 (huge), and ViT-bigG-14 (giant). These models exhibit significant differences in their architectural parameters, ranging from 86M parameters in the baseline ViT-B-32 to over 1B parameters in the ViT-bigG-14. Additionally, they utilize different vision encoder configurations: while ViT-B-32 processes images using 32×32 patches, the larger models employ finer 14×14 patch sizes for increased granularity in visual feature extraction.

As shown in Fig. 11, we first analyze the semantic reward signals generated by different CLIP model variants through a representative case study. The sequence of BEVs in Fig. 11 (a) depicts a critical safety situation where the agent vehicle experiences a side collision with another vehicle. The reward curves in Fig. 11 (b) reveal a clear correlation between model size and reward signal quality. The smallest model, ViT-B-32, produces relatively flat rewards fluctuating around -0.02, indicating poor sensitivity to the collision event. In comparison, ViT-bigG-14 demonstrates remarkably superior performance among all variants, generating the most distinctive and interpretable reward pattern with a sharp decline to -0.14 during the collision phase from steps 30 to 50 before gradually recovering as the vehicles separate. The significant enhancement in semantic reward signal discrimination observed in larger models, particularly exemplified by ViT-bigG-14, indicates that increased model capacity enables more sophisticated scene understanding capabilities and consequently generates more effective learning signals for reinforcement learning agents.

We further evaluate how different CLIP model sizes affect the performance of RL agents. Fig. 12 presents a comprehensive comparison for four variants: VLM-RL-B, VLM-RL-L, VLM-RL-H, and VLM-RL-bigG. The results reveal interesting trade-offs between safety and efficiency across model scales. While VLM-RL-H achieves the lowest collision rate, its low average speed of around 2 km/h suggests that the agent fails to learn basic navigation skills, essentially prioritizing safety at the complete expense of functionality. Conversely, VLM-RL-L exhibits the highest average speed among all variants but at the cost of the highest collision rate, indicating an unsafe bias toward aggressive driving. VLM-RL-B shows comparable performance to VLM-RL-bigG in terms of completed routes and total

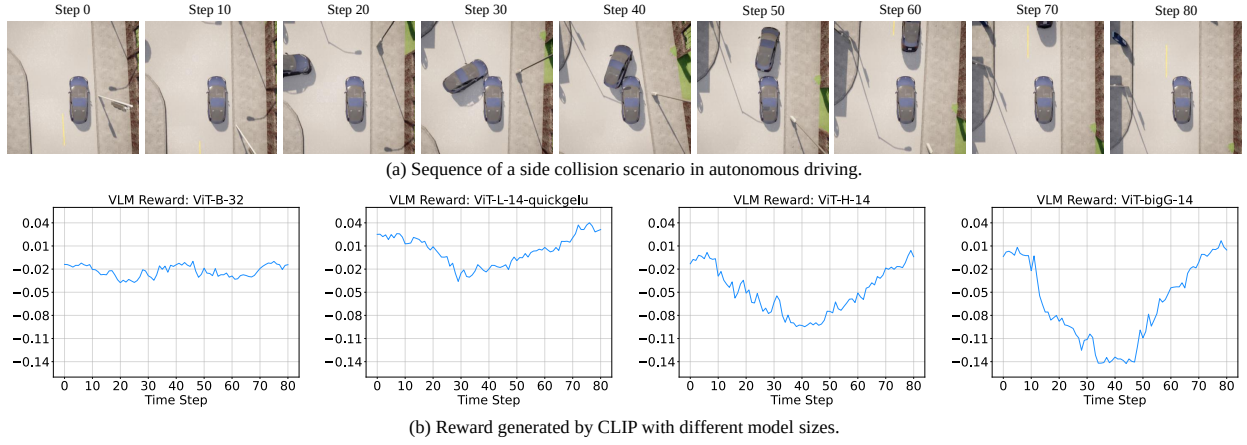


Figure 11: Base rewards with different CLIP model sizes using the CLG in a side collision scenario.

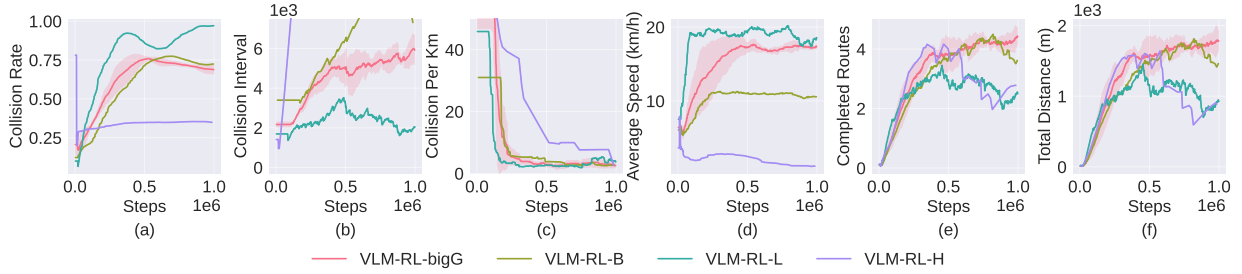


Figure 12: Performance comparison using different CLIP model sizes during training.

distance traveled, but operates at significantly lower speeds, suggesting less efficient navigation. Among all variants, VLM-RL-bigG demonstrates the most balanced performance, maintaining a moderate collision rate while achieving efficient navigation as evidenced by its competitive speed, route completion, and travel distance metrics. These results indicate that the increased model capacity of ViT-bigG-14 enables the agent to better balance the complex trade-offs between safety and efficiency in autonomous driving.

5.7. Generalization

Generalization capability is crucial for autonomous driving systems, as they must adapt to diverse environments and conditions beyond their training scenarios. To comprehensively evaluate VLM-RL’s adaptability to diverse scenarios, we first compare our method with two best-performing baseline approaches, i.e., ChatScene-SAC and Revolve, across different towns and varying traffic densities. We also demonstrate the versatility of our approach by extending it to PPO algorithm and comparing with PPO-based baselines, showing that our CLG-based reward design is effective across different RL frameworks.

5.7.1. Different Towns

To evaluate the generalization capability of VLM-RL across different urban and rural environments, we test our model in five distinct towns within the CARLA simulator. As shown in Fig. 5, Town 2 serves as the training environment, while Towns 1, 3, 4, and 5 represent previously unseen environments with varying layouts and road structures. The detailed performance comparison is shown in Tab. 4.

In Town 1, VLM-RL demonstrates exceptional performance with perfect success and route completion rates of 1.00, while achieving the highest average speed of 22.9 km/h and maintaining a remarkably low collision speed of 0.03 km/h. In contrast, both baseline methods show reduced performance with success rates below 0.72 and substantially higher collision speeds above 4.20 km/h. The performance advantage persists in the more challenging Towns 3 and 5, where VLM-RL maintains high success rates of 0.87 and route completion rates above 0.91. Notably, VLM-RL achieves significantly lower collision speeds compared to the baselines, particularly evident in Town 5 where Revolve exhibits a high collision speed of 5.35 km/h versus VLM-RL’s 0.46 km/h. Town 4 presents the most challenging scenario with longer routes, as reflected in the total driving distances exceeding 12000m. In this environment, while ChatScene-SAC achieves marginally better route completion and success rates, VLM-RL maintains superior driving efficiency with the highest average speed of 22.0 km/h and significantly better safety performance, recording a collision speed of 2.15 km/h compared to 6.85 km/h and 10.0 km/h of the baselines.

These results demonstrate that VLM-RL successfully generalizes to diverse driving environments without additional training. The consistent performance advantages across different towns validate that our CLG-based and

Table 4: Generalization performance across different Towns. Mean and standard deviation over 3 seeds. The best results are marked in **bold**.

Town	Model	AS $\uparrow$	RC $\uparrow$	TD $\uparrow$	CS $\downarrow$	SR $\uparrow$
Town 1	ChatScene-SAC	18.2 $\pm$ 0.25	0.83 $\pm$ 0.09	4408.8 $\pm$ 671.0	4.50 $\pm$ 2.84	0.67 $\pm$ 0.17
	Revolve	18.9 $\pm$ 0.28	0.84 $\pm$ 0.07	4458.7 $\pm$ 449.5	4.27 $\pm$ 2.00	0.72 $\pm$ 0.17
	VLM-RL	22.9 $\pm$ 0.63	1.00 $\pm$ 0.00	5697.6 $\pm$ 0.79	0.03 $\pm$ 0.05	1.00 $\pm$ 0.00
Town 2	ChatScene-SAC	17.7 $\pm$ 0.12	0.88 $\pm$ 0.03	1763.2 $\pm$ 90.9	1.18 $\pm$ 0.46	0.73 $\pm$ 0.05
	Revolve	18.4 $\pm$ 0.03	0.92 $\pm$ 0.11	1915.3 $\pm$ 248.3	1.53 $\pm$ 2.16	0.83 $\pm$ 0.24
	VLM-RL	19.3 $\pm$ 1.29	0.97 $\pm$ 0.03	2028.2 $\pm$ 96.6	0.02 $\pm$ 0.03	0.93 $\pm$ 0.04
Town 3	ChatScene-SAC	17.8 $\pm$ 0.22	0.85 $\pm$ 0.04	3436.5 $\pm$ 355.4	1.60 $\pm$ 0.44	0.77 $\pm$ 0.12
	Revolve	18.5 $\pm$ 0.41	0.75 $\pm$ 0.16	2979.3 $\pm$ 649.5	5.22 $\pm$ 0.77	0.53 $\pm$ 0.21
	VLM-RL	21.7 $\pm$ 0.55	0.91 $\pm$ 0.07	3757.8 $\pm$ 248.0	1.14 $\pm$ 1.54	0.87 $\pm$ 0.09
Town 4	ChatScene-SAC	18.1 $\pm$ 0.32	0.82 $\pm$ 0.08	14139.0 $\pm$ 1650.9	6.85 $\pm$ 1.92	0.70 $\pm$ 0.08
	Revolve	17.0 $\pm$ 2.88	0.63 $\pm$ 0.10	9874.4 $\pm$ 2132.1	10.0 $\pm$ 1.87	0.40 $\pm$ 0.08
	VLM-RL	22.0 $\pm$ 3.69	0.80 $\pm$ 0.17	12684.1 $\pm$ 3608.5	2.15 $\pm$ 1.59	0.70 $\pm$ 0.28
Town 5	ChatScene-SAC	18.4 $\pm$ 0.06	0.85 $\pm$ 0.04	2826.3 $\pm$ 350.6	2.14 $\pm$ 1.34	0.73 $\pm$ 0.09
	Revolve	19.0 $\pm$ 0.63	0.78 $\pm$ 0.06	2671.6 $\pm$ 231.4	5.35 $\pm$ 1.95	0.53 $\pm$ 0.09
	VLM-RL	22.9 $\pm$ 0.86	0.93 $\pm$ 0.03	3322.5 $\pm$ 203.4	0.46 $\pm$ 0.54	0.87 $\pm$ 0.05

hierarchical reward design captures fundamental driving principles rather than overfitting to specific environmental features. This robust generalization capability is particularly crucial for real-world autonomous driving applications where vehicles must navigate diverse and previously unseen environments.

5.7.2. Different Traffic Densities

To evaluate the robustness of VLM-RL under varying traffic conditions, we test our model in three traffic density settings as shown in Tab. 5: empty scenarios with no other vehicles, regular scenarios which is our default setting with 20 autopilot vehicles, and dense scenarios with 40 autopilot vehicles.

In empty scenarios, all methods achieve perfect success and route completion rates of 1.0, with zero collision speeds, demonstrating their fundamental capability in obstacle-free environments. However, VLM-RL exhibits superior driving efficiency with an average speed of 23.8 km/h, significantly higher than ChatScene-SAC at 18.0 km/h and Revolve at 18.6 km/h. In dense traffic scenarios, the performance differences become more pronounced in terms of safety. While Revolve achieves marginally better efficiency metrics with the highest average speed of 18.3 km/h and route completion of 0.89, its collision speed increases significantly to 3.53 km/h. Similarly, ChatScene-SAC shows degraded safety performance with a collision speed of 2.71 km/h. In contrast, VLM-RL maintains excellent safety with a collision speed of 0.11 km/h while achieving comparable route completion of 0.87 and the highest success rate of 0.80. These results demonstrate an important characteristic of VLM-RL. As traffic density increases, the model adapts by prioritizing safety over speed, demonstrating intelligent risk-aware behavior. This adaptive balance between efficiency and safety is particularly valuable for real-world autonomous driving systems that must operate safely across diverse traffic conditions.

5.7.3. Different RL Algorithms

We extend our VLM-RL framework to the PPO algorithm to demonstrate its compatibility with different RL algorithms. Fig. 13 compares the training performance of VLM-RL-PPO with other PPO-based baselines including TIRL-PPO, ASAP-RL-PPO, and ChatScene-PPO. The results show that our CLG-based hierarchical reward design maintains its effectiveness when implemented with PPO.

As illustrated in Fig. 13, VLM-RL-PPO demonstrates superior performance across most metrics. While TIRL-PPO shows low collision rates, its average speed and completed routes remain close to zero throughout training, indicating the agent fails to learn meaningful driving behaviors. ASAP-RL-PPO achieves more stable training but shows limited progress in route completion and total distance traveled. ChatScene-PPO exhibits better driving capabilities with moderate average speed but struggles to maintain consistent performance, as shown by the fluctuating collision rates and limited route completion. Most notably, VLM-RL-PPO shows remarkable learning progress in

Table 5: Generalization performance across different traffic densities. Mean and standard deviation over 3 seeds. The best results are marked in **bold**.

Traffic Density	Model	AS $\uparrow$	RC $\uparrow$	TD $\uparrow$	CS $\downarrow$	SR $\uparrow$
Empty	ChatScene-SAC	18.0 $\pm$ 0.18	1.0 $\pm$ 0.0	2064.1 $\pm$ 4.81	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0
	Revolve	18.6 $\pm$ 0.39	1.0 $\pm$ 0.0	2105.5 $\pm$ 21.0	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0
	VLM-RL	23.8 $\pm$ 0.29	1.0 $\pm$ 0.0	2113.9 $\pm$ 0.67	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0
Regular	ChatScene-SAC	17.7 $\pm$ 0.12	0.88 $\pm$ 0.03	1763.2 $\pm$ 90.9	1.18 $\pm$ 0.46	0.73 $\pm$ 0.05
	Revolve	18.4 $\pm$ 0.03	0.92 $\pm$ 0.11	1915.3 $\pm$ 248.3	1.53 $\pm$ 2.16	0.83 $\pm$ 0.24
	VLM-RL	19.3 $\pm$ 1.29	0.97 $\pm$ 0.03	2028.2 $\pm$ 96.6	0.02 $\pm$ 0.03	0.93 $\pm$ 0.04
Dense	ChatScene-SAC	17.3 $\pm$ 0.15	0.85 $\pm$ 0.06	1734.2 $\pm$ 92.3	2.71 $\pm$ 0.65	0.77 $\pm$ 0.12
	Revolve	18.3 $\pm$ 0.31	0.89 $\pm$ 0.06	1861.0 $\pm$ 144.3	3.53 $\pm$ 0.52	0.73 $\pm$ 0.05
	VLM-RL	16.1 $\pm$ 1.00	0.87 $\pm$ 0.06	1819.0 $\pm$ 166.6	0.11 $\pm$ 0.10	0.80 $\pm$ 0.08

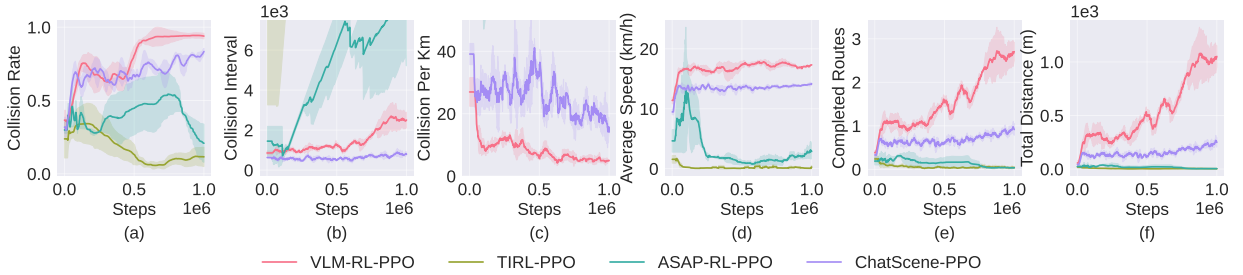


Figure 13: Performance comparison among PPO-based methods.

navigation capabilities. The number of completed routes demonstrates a strong upward trend throughout training, reaching approximately 2.5 routes compared to less than 1 route for other methods. Similarly, the total distance traveled exhibits substantial and consistent growth, ultimately achieving around 1000m, while other PPO-based methods remain below 300m. This improvement in both metrics indicates that VLM-RL-PPO effectively learns to navigate complex environments and complete driving tasks. Meanwhile, it maintains a stable average speed of around 15 km/h while successfully managing collision risks. These results validate that our CLG-based hierarchical reward design can be effectively integrated with different RL algorithms, suggesting the broader applicability of our approach in safe driving tasks.

5.8. Visualization of VLM Semantic Rewards

To better understand the effectiveness and limitations of CLIP-derived semantic rewards, we visualize three representative failure cases and analyze how the positive, negative, and CLG capture different driving behaviors. Fig. 14 shows image sequences and their corresponding semantic similarity scores for three challenging scenarios.

In the rear-end collision scenario shown in Fig. 14 (a), we observe that the positive goal similarity score gradually decreases as the ego vehicle approaches the leading vehicle, while the negative goal similarity increases. This pattern aligns with our design intention, where unsafe behaviors should result in lower positive goal similarity and higher negative goal similarity. The final contrasting score shows a clear downward trend, correctly reflecting the undesirable nature of the rear-end collision behavior. The side collision case illustrated in Fig. 14 (b) reveals another interesting pattern. The positive goal similarity fails to show a significant decrease during the collision period from steps 30 to 45, suggesting that VLM sometimes struggles to capture safety violations from the positive perspective alone. However, the negative goal similarity exhibits a pronounced peak during the collision, leading to an appropriate dip in the contrasting score that properly penalizes this unsafe behavior. The out-of-road scenario presented in Fig. 14 (c) highlights a limitation of purely VLM-based semantic rewards. While the positive goal similarity decreases as the vehicle deviates from the road, the negative goal similarity also shows a declining trend. This results in a relatively flat contrasting score that fails to adequately penalize the out-of-road behavior. This limitation stems from our language goals primarily focusing on collision-related behaviors, lacking explicit consideration of road boundary violations.

These observations highlight both the potential and limitations of VLM-derived semantic rewards. While the CLG can effectively capture many unsafe behaviors, it may fail to provide appropriate learning signals for certain

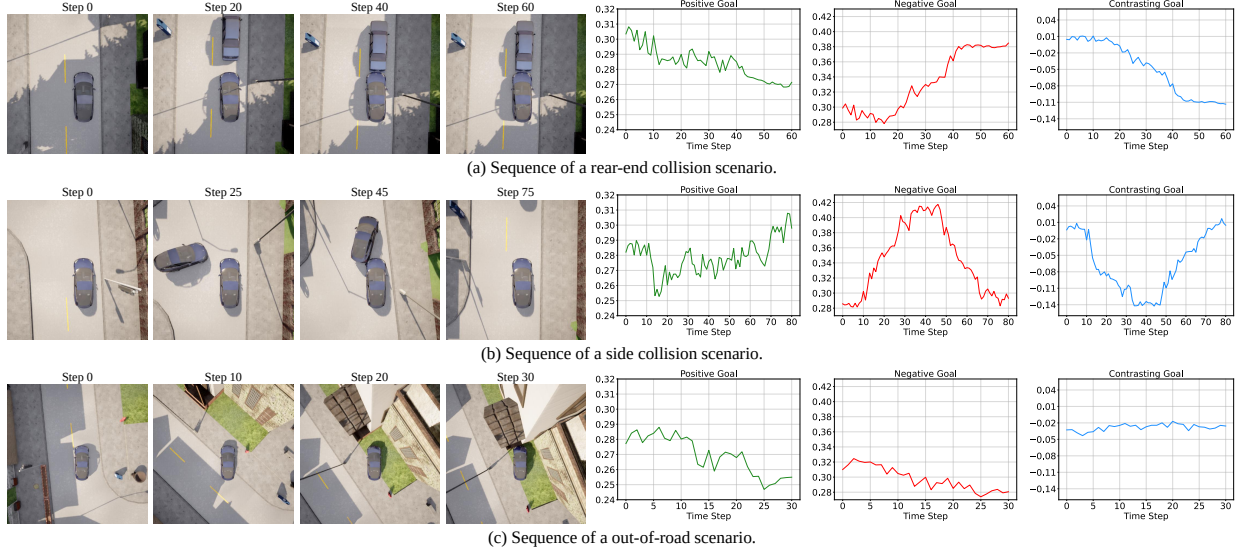


Figure 14: CLIP rewards using different CLGs in different scenarios.

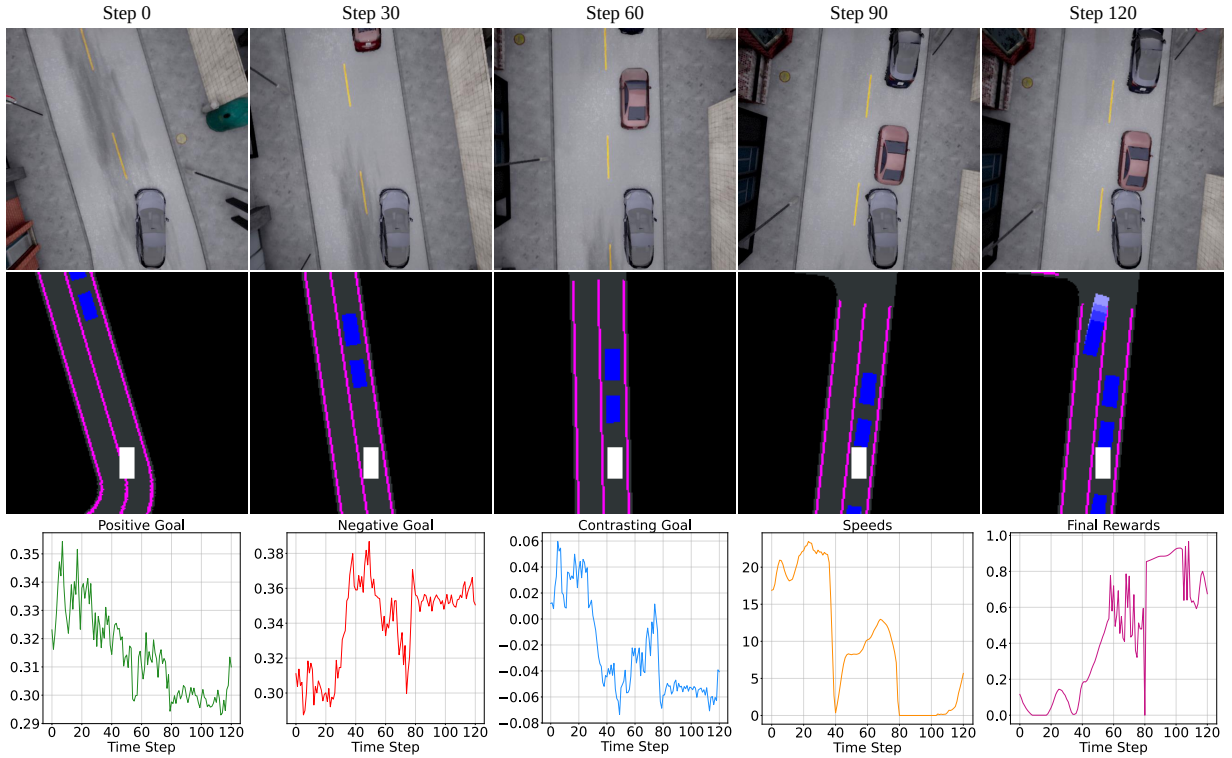


Figure 15: CLIP rewards using different CLGs in different scenarios.

scenarios, particularly those not explicitly described in the language goals. This underscores the necessity of our hierarchical reward synthesis approach, which combines this high-level semantic understanding with low-level vehicle state information to provide more comprehensive and reliable reward signals.

5.9. Analysis of Hierarchical Reward Synthesis

Figs. 15 and 16 illustrate how our hierarchical reward synthesis approach combines VLM-derived semantic rewards with vehicle state information to generate comprehensive final rewards. The scenario in Fig. 15 shows a collision situation where the ego vehicle approaches a stationary vehicle ahead. The first row presents the RGB images, while the second row shows the corresponding semantic segmentation maps. In the third row, we observe how different reward components evolve throughout the scenario. As the ego vehicle approaches the leading vehicle, the positive goal similarity steadily decreases while the negative goal similarity increases, resulting in a declining contrasting goal score. Similar to Fig. 14 (a), this pattern effectively captures the increasing risk of collision. The speed profile reveals several distinct phases of the ego vehicle’s behavior. Before step 40, the ego vehicle maintains

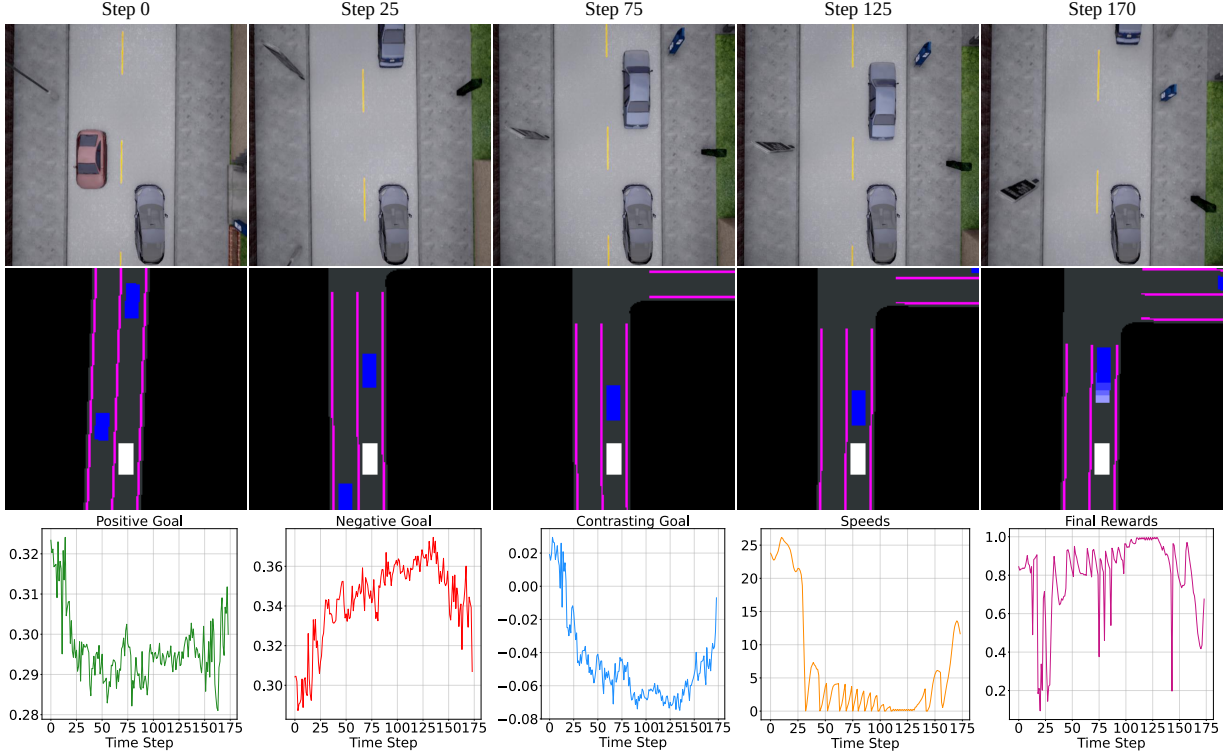


Figure 16: CLIP rewards using different CLGs in different scenarios.

a relatively high speed despite approaching the stationary vehicle, a behavior that our VLM identifies as potentially unsafe. Consequently, the final reward remains close to zero during this period, effectively penalizing this aggressive driving behavior. Between steps 40 and 80, the ego vehicle significantly reduces its speed in response to the potential collision risk. This cautious behavior is rewarded with an increased final reward, demonstrating how our reward synthesis effectively encourages appropriate speed adjustments. During steps 80-100, when the ego vehicle comes to a complete stop behind the leading vehicle, the final reward reaches its highest values, validating that our reward function correctly identifies this as the desirable behavior in this scenario. However, after step 110, when the ego vehicle unexpectedly begins to move again despite the continued presence of the obstacle, we observe a sharp decline in the final reward, properly penalizing this undesired behavior.

Fig. 16 presents a successful driving scenario that demonstrates how our reward effectively guides the RL agent through complex, multi-phase driving situations. The scenario involves approaching a stationary vehicle, maintaining a safe following distance, and resuming motion when the leading vehicle departs. The evolution of VLM-derived semantic rewards effectively captures the changing nature of the scene. The positive goal similarity initially decreases as the ego vehicle approaches the stationary vehicle, then increases after the leading vehicle departs and the road becomes clear. Conversely, the negative goal similarity rises during the approach phase and declines after the leading vehicle’s departure. The contrasting goal synthesizes these patterns, providing a clear learning signal that aligns with safe driving behaviors. The speed profile reveals several distinct driving phases that showcase the agent’s learned behavior. Initially, the ego vehicle travels at a relatively high speed of approximately 25 km/h when the road is clear. Between steps 25 and 125, as it detects and responds to the stationary vehicle ahead, the speed gradually reduces to around 5 km/h before coming to a complete stop. The final reward during this period remains consistently high, with minor fluctuations around step 25 due to a slightly delayed braking response. After step 125, when the leading vehicle resumes motion, the ego vehicle demonstrates appropriate acceleration behavior, though we observe some reward fluctuations around step 160 due to momentarily aggressive acceleration that temporarily reduces the safety margin.

The cases in Figs. 15 and 16 study highlight how our hierarchical reward synthesis successfully guides the agent through different driving phases while maintaining safety. The final reward effectively balances the semantic understanding of the scene captured by the VLM-derived rewards with concrete vehicle states, encouraging behavior that is both efficient and safe. Our approach demonstrates robustness in handling dynamic situations, appropriately transitioning between different driving modes while maintaining high reward values for safe behavior and penalizing potentially risky actions through reward modulation.

6. Conclusions and Future Work

This paper presented VLM-RL, a unified framework that integrates pre-trained Vision-Language Models (VLMs) with Reinforcement Learning (RL) for safe autonomous driving. The key contributions of this work lie in addressing the challenges of reward design by leveraging the semantic understanding capabilities of VLMs. Specifically, we introduced the Contrasting language goal (CLG)-as-reward paradigm, which utilizes both positive and negative language goals to generate semantic rewards. We further introduced a hierarchical reward synthesis method that combines CLG-based rewards with vehicle state information, improving reward stability and offering a more comprehensive learning signal. To mitigate computational challenges, a batch-processing technique was employed, enabling efficient reward computation during training. Extensive experiments conducted in the CARLA simulator demonstrated the efficacy of the VLM-RL. Compared to state-of-the-art baselines, VLM-RL achieved significant improvements in safety, efficiency, and generalization to unseen driving scenarios. Notably, VLM-RL reduced collision rates by 10.5%, increased route completion rates by 104.6%, and exhibited robust performance across diverse traffic environments. These results validate the potential of integrating VLMs with RL to develop more reliable and context-aware autonomous driving policies.

While VLM-RL shows promising results, several avenues for future work remain. First, improving the computational efficiency of VLM inference during training and deployment remains a challenge. Techniques such as model distillation or quantization could be investigated to reduce inference latency. Second, expanding the range of driving tasks and scenarios to include interactions with pedestrians, cyclists, and traffic signals—particularly traffic lights, which are not currently considered—could significantly enhance the framework’s robustness and realism. Additionally, integrating human-in-the-loop feedback for refining CLG-based rewards dynamically may provide additional adaptability and personalization, further aligning driving policies with human preferences. In summary, VLM-RL provides a robust and scalable solution for reward design in RL-based autonomous driving, offering a promising direction for achieving safer and more generalizable driving policies. Continued research in this area has the potential to bridge the gap between simulation-based learning and real-world deployment, advancing the field of autonomous driving toward human-level safety and reliability.

Acknowledgment

This work was supported by the University of Wisconsin-Madison’s Center for Connected and Automated Transportation (CCAT), a part of the larger CCAT consortium, a USDOT Region 5 University Transportation Center funded by the U.S. Department of Transportation, Award #69A3552348305. The contents of this paper reflect the views of the authors, who are responsible for the facts and the accuracy of the data presented herein, and do not necessarily reflect the official views or policies of the sponsoring organization.

Appendix A. Robustness of the CLG-as-Reward Paradigm

In this appendix, we show that the CLG reward defined in Eq. (7) also enhances robustness against uncertainties or adversarial perturbations. We first establish a lemma concerning the Lipschitz continuity of the cosine similarity measure. We then prove that R_{CLG} inherits this Lipschitz continuity under suitable assumptions. Finally, we demonstrate how this continuity, combined with the presence of both positive and negative goals, improves the robustness of the learned policy.

Lemma 1 (Lipschitz Continuity of Cosine Similarity). *Let $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^k$ be unit vectors (i.e., $\|\mathbf{u}\| = \|\mathbf{v}\| = \|\mathbf{w}\| = 1$). Then, the cosine similarity function is Lipschitz continuous with Lipschitz constant $L = 1$. In particular, for any such vectors*

$$|\text{sim}(\mathbf{u}, \mathbf{w}) - \text{sim}(\mathbf{v}, \mathbf{w})| \leq \|\mathbf{u} - \mathbf{v}\| \quad (\text{A.1})$$

Proof 2. *Since $\mathbf{u}, \mathbf{v}, \mathbf{w}$ are unit vectors, the cosine similarity reduces to the inner product:*

$$\text{sim}(\mathbf{u}, \mathbf{w}) = \mathbf{u}^\top \mathbf{w} \quad (\text{A.2})$$

Then,

$$|\text{sim}(\mathbf{u}, \mathbf{w}) - \text{sim}(\mathbf{v}, \mathbf{w})| = |\mathbf{u}^\top \mathbf{w} - \mathbf{v}^\top \mathbf{w}| \quad (\text{A.3})$$

$$= |(\mathbf{u} - \mathbf{v})^\top \mathbf{w}| \quad (\text{A.4})$$

$$\leq \|\mathbf{u} - \mathbf{v}\| \cdot \|\mathbf{w}\| \quad (\text{by Cauchy-Schwarz inequality}) \quad (\text{A.5})$$

$$= \|\mathbf{u} - \mathbf{v}\| \cdot 1 = \|\mathbf{u} - \mathbf{v}\| \quad (\text{A.6})$$

Theorem 2 (Lipschitz Continuity of the CLG Reward Function). Suppose the VLM encoders produce unit-length embeddings. Under this assumption, R_{CLG} is Lipschitz continuous with Lipschitz constant $L = 1$. Specifically, for any two states $s_1, s_2 \in \mathcal{S}$:

$$|R_{CLG}(s_1) - R_{CLG}(s_2)| \leq |\mathbf{v}_1 - \mathbf{v}_2| \quad (\text{A.7})$$

where $\mathbf{v}_i = \text{VLM}_I(\psi(s_i))$.

Proof 3. Recall the definition of R_{CLG} from Eq. (7):

$$R_{CLG}(s) = \alpha \cdot \text{sim}(\mathbf{v}, \mathbf{v}_{pos}) - \beta \cdot \text{sim}(\mathbf{v}, \mathbf{v}_{neg}) \quad (\text{A.8})$$

with $\alpha, \beta > 0$ and $\alpha + \beta = 1$, and $\mathbf{v} = \text{VLM}_I(\psi(s))$.

For any s_1, s_2 , let $\mathbf{v}_1 = \text{VLM}_I(\psi(s_1))$ and $\mathbf{v}_2 = \text{VLM}_I(\psi(s_2))$. Then:

$$|R_{CLG}(s_1) - R_{CLG}(s_2)| = |\alpha(\text{sim}(\mathbf{v}_1, \mathbf{v}_{pos}) - \text{sim}(\mathbf{v}_2, \mathbf{v}_{pos})) - \beta(\text{sim}(\mathbf{v}_1, \mathbf{v}_{neg}) - \text{sim}(\mathbf{v}_2, \mathbf{v}_{neg}))| \quad (\text{A.9})$$

$$\leq \alpha|\text{sim}(\mathbf{v}_1, \mathbf{v}_{pos}) - \text{sim}(\mathbf{v}_2, \mathbf{v}_{pos})| + \beta|\text{sim}(\mathbf{v}_1, \mathbf{v}_{neg}) - \text{sim}(\mathbf{v}_2, \mathbf{v}_{neg})| \quad (\text{A.10})$$

By Lemma 1, since embeddings are unit norm,

$$|\text{sim}(\mathbf{v}_1, \mathbf{v}_g) - \text{sim}(\mathbf{v}_2, \mathbf{v}_g)| \leq |\mathbf{v}_1 - \mathbf{v}_2| \quad (\text{A.11})$$

for $g \in \{\text{pos}, \text{neg}\}$. Substituting this into the inequality above:

$$|R_{CLG}(s_1) - R_{CLG}(s_2)| \leq \alpha|\mathbf{v}_1 - \mathbf{v}_2| + \beta|\mathbf{v}_1 - \mathbf{v}_2| \quad (\text{A.12})$$

$$= (\alpha + \beta)|\mathbf{v}_1 - \mathbf{v}_2| = |\mathbf{v}_1 - \mathbf{v}_2| \quad (\text{A.13})$$

Thus, R_{CLG} is Lipschitz continuous with $L = 1$.

Remark 1. Lipschitz continuity ensures that small changes in the observation lead to small changes in the reward. This property helps stabilize RL training by reducing variance in gradient estimates and making the learning process more predictable (Bhandari et al., 2018).

Theorem 3 (Robustness Enhancement of the CLG Reward). Assume that both the VLM encoder and the observation function ψ are Lipschitz continuous. Let L_{VLM} and L_ψ denote their respective Lipschitz constants. Consider a perturbed state $s' = s + \delta_s$, where δ_s is a small perturbation. Then there exists a constant $K = L_{VLM}L_\psi$ such that:

$$|R_{CLG}(s) - R_{CLG}(s')| \leq K|\delta_s| \quad (\text{A.14})$$

This shows that the change in R_{CLG} under perturbations is linearly bounded by the magnitude of the perturbation, making the policy more robust to adversarial or uncertain disturbances.

Proof 4. Define $\mathbf{v}_s = \text{VLM}_I(\psi(s))$ and $\mathbf{v}_{s'} = \text{VLM}_I(\psi(s'))$. If VLM_I is L_{VLM} -Lipschitz and ψ is L_ψ -Lipschitz, we have:

$$|\mathbf{v}_s - \mathbf{v}_{s'}| \leq L_{VLM}|\psi(s) - \psi(s')| \leq L_{VLM}L_\psi|\delta_s| \quad (\text{A.15})$$

From Thm 2, it follows that:

$$|R_{CLG}(s) - R_{CLG}(s')| \leq |\mathbf{v}_s - \mathbf{v}_{s'}| \quad (\text{A.16})$$

Combining these inequalities:

$$|R_{CLG}(s) - R_{CLG}(s')| \leq L_{VLM}L_\psi|\delta_s| = K|\delta_s| \quad (\text{A.17})$$

This shows that the CLG reward fluctuation under perturbations is bounded. Now consider that in adversarial or uncertain environments, perturbations may selectively decrease similarity to the positive goal or increase similarity to the negative goal. Since R_{CLG} includes both positive and negative goals, it provides a form of redundancy: if a perturbation maliciously affects one similarity, the other can partially compensate. A carefully chosen perturbation could drastically alter a single-target reward without this dual structure. Thus, the dual-goal structure of R_{CLG} inherently enhances robustness.

Corollary 1. A policy trained with R_{CLG} is more robust in adversarial or uncertain environments than one trained using only positive or only negative goals. The incorporation of both goals makes it harder for perturbations to significantly degrade the reward, thereby maintaining a stable and safe policy.

Proof 5. The corollary follows directly from Theorem 3 and the preceding analysis. If only a single goal (positive or negative) is used, a perturbation can be designed to specifically degrade that similarity score, causing a significant shift in the reward. However, R_{CLG} combines both, making it harder for a perturbation to simultaneously degrade both terms advantageously, thus maintaining reward stability and robustness.

Remark 2. These theoretical findings align with empirical results, where policies trained with R_{CLG} show improved stability and resilience in complex, dynamic, or adversarial driving scenarios.

Appendix B. Workflow of Hierarchical Reward Synthesis

Algorithm 1 Hierarchical Reward Synthesis

Input: Observation o , vehicle state s , positive language goal l_{pos} , negative language goal l_{neg} , weighting factors α, β (where $\alpha + \beta = 1$), and predefined thresholds $\theta_{\min}, \theta_{\max}$

- 1: **Compute Semantic Reward:**
- 2: Compute image embedding: $\mathbf{v}_o \leftarrow \text{VLM}_I(o)$
- 3: Compute positive goal embedding: $\mathbf{v}_{\text{pos}} \leftarrow \text{VLM}_L(l_{\text{pos}})$
- 4: Compute negative goal embedding: $\mathbf{v}_{\text{neg}} \leftarrow \text{VLM}_L(l_{\text{neg}})$
- 5: Compute positive similarity using Eq. (5): $s_{\text{pos}} \leftarrow \text{sim}(\mathbf{v}_o, \mathbf{v}_{\text{pos}})$
- 6: Compute negative similarity using Eq. (5): $s_{\text{neg}} \leftarrow \text{sim}(\mathbf{v}_o, \mathbf{v}_{\text{neg}})$
- 7: Compute semantic reward using Eq. (7): $r_{\text{CLG}} \leftarrow \alpha \cdot s_{\text{pos}} - \beta \cdot s_{\text{neg}}$
- 8: Normalize r_{CLG} to $[0, 1]$ using Eq. (10):
- 9: $r'_{\text{CLG}} \leftarrow 1 - \text{clip}(r_{\text{CLG}}, \theta_{\min}, \theta_{\max})$
- 10: **Compute Vehicle State Rewards:**
- 11: Compute target speed: $v_{\text{target}} \leftarrow r'_{\text{CLG}} \cdot v_{\max}$
- 12: Compute speed alignment (as described in Section 4):
- 13: $r_{\text{speed}} \leftarrow 1 - \frac{|v - v_{\text{target}}|}{v_{\max}}$
- 14: Compute $f_{\text{center}}(s)$ based on lateral deviation to lane center
- 15: Compute $f_{\text{angle}}(s)$ based on heading alignment
- 16: Compute $f_{\text{stability}}(s)$ based on lateral position stability
- 17: **Synthesize Reward:**
- 18: Compute final synthesis using Eq. (11):
- 19: $r_{\text{synthesis}} \leftarrow r_{\text{speed}} \times f_{\text{center}} \times f_{\text{angle}} \times f_{\text{stability}}$
- 20: **return** $r_{\text{synthesis}}$

Appendix C. Convergence of the Synthesis Reward Function

Theorem 4 (Convergence of Policy Optimization). *Under standard assumptions for the SAC algorithm (Haarnoja et al., 2018), policy optimization with the synthesis reward $R_{\text{synthesis}}(s)$ defined in Eq. (11) converges to a local optimum.*

Proof 6. *The SAC algorithm seeks to maximize the expected discounted return, augmented by an entropy term:*

$$J(\pi_\phi) = \mathbb{E}_{\pi_\phi} \left[\sum_{t=0}^T \gamma^t (R_{\text{synthesis}}(s_t) + \alpha \mathcal{H}(\pi_\phi(\cdot|s_t))) \right], \quad (\text{C.1})$$

where $\alpha > 0$ is the temperature parameter controlling the trade-off between return and entropy, $\gamma \in [0, 1)$ is the discount factor, and $\mathcal{H}(\pi_\phi(\cdot|s_t))$ denotes the entropy of the policy π_ϕ at state s_t .

By construction, the synthesis reward function $R_{\text{synthesis}}(s)$ in Eq. (11) is a product of several bounded and continuous components:

- $r'_t{}^{\text{CLG}}$ is bounded due to the clipping operation in its definition.
- Each vehicle state factor ($r_{\text{speed}}(s)$, $f_{\text{center}}(s)$, $f_{\text{angle}}(s)$, $f_{\text{stability}}(s)$) is designed to be bounded and continuous. In detail, $r_{\text{speed}}(s) \in [0, 1]$, $f_{\text{center}}(s) \in [0, 1]$, $f_{\text{angle}}(s) \in [0, 1]$, $f_{\text{stability}}(s) \in [0, 1]$. Thus, $R_{\text{synthesis}}(s)$ is itself bounded and continuous, and specifically $R_{\text{synthesis}}(s) \in [0, 1]$.

Given $R_{\text{synthesis}}(s)$ is bounded and continuous, and assuming that the policy π_ϕ and the value function approximators are expressive enough (e.g., using neural networks with sufficient capacity), the SAC algorithm satisfies the convergence properties as established in (Haarnoja et al., 2018).

Therefore, under these standard assumptions, the policy optimization using the synthesis reward function $R_{\text{synthesis}}(s)$ will converge to a local optimum.

Appendix D. Stability of the Synthesis Reward Function

Theorem 5 (Lipschitz Continuity of the Synthesis Reward Function). *The synthesis reward function $R_{\text{synthesis}}(s)$ is Lipschitz continuous with respect to the state s .*

Proof 7. To show Lipschitz continuity, we need to show that there exists a constant $L > 0$ such that for any two states s_1 and s_2 :

$$|R_{\text{synthesis}}(s_1) - R_{\text{synthesis}}(s_2)| \leq L|s_1 - s_2|. \quad (\text{D.1})$$

The synthesis reward $R_{\text{synthesis}}(s)$ is computed as:

$$R_{\text{synthesis}}(s) = r_{\text{speed}}(s) \cdot f_{\text{center}}(s) \cdot f_{\text{angle}}(s) \cdot f_{\text{stability}}(s), \quad (\text{D.2})$$

where $r_{\text{speed}}(s)$, $f_{\text{center}}(s)$, $f_{\text{angle}}(s)$, and $f_{\text{stability}}(s)$ are components that evaluate different aspects of the driving task. To prove Lipschitz continuity of $R_{\text{synthesis}}(s)$, we analyze the individual components.

The speed alignment reward $r_{\text{speed}}(s) = 1 - \frac{|v(s) - v_{\text{target}}(s)|}{v_{\text{max}}}$ is Lipschitz continuous because both the current speed $v(s)$ and target speed $v_{\text{target}}(s) = r_t^{\text{CLG}} \cdot v_{\text{max}}$ are continuous functions of the state s . Additionally, the absolute value function and normalization by v_{max} preserve Lipschitz continuity. Similarly, the lane centering factor $f_{\text{center}}(s)$ and the heading alignment factor $f_{\text{angle}}(s)$ are both continuous functions with respect to the state s . The distance stability term $f_{\text{stability}}(s)$ is also continuous due to its dependence on the positions of the ego vehicle.

Since each component of $R_{\text{synthesis}}(s)$ is Lipschitz continuous and bounded within $[0, 1]$, their product is also Lipschitz continuous according to the properties of bounded Lipschitz functions. Thus, $R_{\text{synthesis}}(s)$ satisfies the required Lipschitz condition.

Remark 3. Lipschitz continuity of the reward function ensures that small changes in the state lead to small changes in the reward. This property contributes to the stability of the learning process by preventing large fluctuations in the gradient estimates during policy optimization (Bhandari et al., 2018).

Appendix E. Workflow of Policy Training with Batch-Processing

Algorithm 2 Policy Training with Batch-Processing and SAC

Input: Initial policy parameters ϕ , Q-function parameters θ , empty replay buffer $\mathcal{D}$, batch size B , total training steps T , VLM encoders VLM_I , VLM_L , language goals l_{pos} , l_{neg} , weighting factors α , β (where $\alpha + \beta = 1$), and predefined thresholds $\theta_{\min}$, $\theta_{\max}$

- 1: **Precompute Goal Embeddings:**
- 2: $\mathbf{v}_{\text{pos}} \leftarrow \text{VLM}_L(l_{\text{pos}})$
- 3: $\mathbf{v}_{\text{neg}} \leftarrow \text{VLM}_L(l_{\text{neg}})$
- 4: **for** $t = 1$ to T **do**
- 5: Observe state s_t and raw observation o_t
- 6: Select action $a_t \sim \pi_\phi(a_t|s_t)$
- 7: Execute a_t in environment
- 8: Receive reward r_t (initially set to 0), next state s_{t+1} , and observation o_{t+1}
- 9: Store transition $(o_t, s_t, a_t, r_t, o_{t+1}, s_{t+1})$ in $\mathcal{D}$
- 10: **if** time to update **then**
- 11: Sample mini-batch $\{(o_i, s_i, a_i, r_i, o_{i+1}, s_{i+1})\}_{i=1}^B$ from $\mathcal{D}$
- 12: **for** each transition in mini-batch **do**
- 13: **Compute Semantic Reward:**
- 14: $\mathbf{v}_{o_i} \leftarrow \text{VLM}_I(o_i)$
- 15: $s_{\text{pos}} \leftarrow \text{sim}(\mathbf{v}_{o_i}, \mathbf{v}_{\text{pos}})$
- 16: $s_{\text{neg}} \leftarrow \text{sim}(\mathbf{v}_{o_i}, \mathbf{v}_{\text{neg}})$
- 17: Compute CLG reward using Eq. (7):
- 18: $r_{\text{CLG}} \leftarrow \alpha \cdot s_{\text{pos}} - \beta \cdot s_{\text{neg}}$
- 19: Normalize r_{CLG} using Eq. (10):
- 20: $r_t^{\text{CLG}} = \frac{\text{clip}(r_t^{\text{CLG}}, \theta_{\min}, \theta_{\max}) - \theta_{\min}}{\theta_{\max} - \theta_{\min}}$
- 21: **Compute Vehicle State Rewards:**
- 22: Compute $v_{\text{target}} \leftarrow r_{\text{CLG}} \cdot v_{\max}$
- 23: $r_{\text{speed}} \leftarrow 1 - \frac{|v_i - v_{\text{target}}|}{v_{\max}}$
- 24: Compute f_{center} , f_{angle} , $f_{\text{stability}}$ based on vehicle state
- 25: **Compute Synthesis Reward:**
- 26: Compute $r_{\text{synthesis}}$ using Eq. (11):
- 27: $r_{\text{synthesis}} \leftarrow r_{\text{speed}} \times f_{\text{center}} \times f_{\text{angle}} \times f_{\text{stability}}$
- 28: Update $r_i \leftarrow r_{\text{synthesis}}$ in replay buffer
- 29: **end for**
- 30: **Update Critic Networks:**
- 31: Update Q-function parameters θ by minimizing $J_Q(\theta)$ as in Eq. (16)
- 32: **Update Policy Network:**
- 33: Update policy parameters ϕ by minimizing $J_\pi(\phi)$ as in Eq. (14)
- 34: **Adjust Temperature Parameter:**
- 35: **end if**
- 36: **end for**

References

- Abouelazm, A., Michel, J., Zoellner, J.M., 2024. A review of reward functions for reinforcement learning in the context of autonomous driving. arXiv preprint arXiv:2405.01440 .
- Adeniji, A., Xie, A., Sferrazza, C., Seo, Y., James, S., Abbeel, P., 2023. Language reward modulation for pretraining reinforcement learning. arXiv preprint arXiv:2308.12270 .
- Baumli, K., Singh, S., Behbahani, F., Chan, H., Comanici, G., Flennerhag, S., Gazeau, M., Holsheimer, K., Horgan, D., Laskin, M., et al., 2023. Vision-language models as a source of rewards, in: Second Agent Learning in Open-Endedness Workshop at NeurIPS 2023.
- Bhandari, J., Russo, D., Singal, R., 2018. A finite time analysis of temporal difference learning with linear function approximation, in: Conference on learning theory, PMLR. pp. 1691–1692.
- Cao, Z., Xu, S., Jiao, X., Peng, H., Yang, D., 2022. Trustworthy safety improvement for autonomous driving using reinforcement learning. Transportation research part C: emerging technologies 138, 103656.
- Chen, J., Li, S.E., Tomizuka, M., 2022. Interpretable end-to-end urban autonomous driving with latent deep reinforcement learning. IEEE Transactions on Intelligent Transportation Systems 23.
- Cui, C., Ma, Y., Cao, X., Ye, W., Wang, Z., 2024. Drive as you speak: Enabling human-like interaction with large language models in autonomous vehicles, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 902–909.
- Di, X., Shi, R., 2021. A survey on autonomous vehicle control in the era of mixed-autonomy: From physics-based to ai-guided driving policy learning. Transportation research part C: emerging technologies 125, 103008.
- Feng, S., Sun, H., Yan, X., Zhu, H., Zou, Z., Shen, S., Liu, H.X., 2023. Dense reinforcement learning for safety validation of autonomous vehicles. Nature 615, 620–627.

- Fu, Y., Zhang, H., Wu, D., Xu, W., Boulet, B., 2024. Furl: Visual-language models as fuzzy rewards for reinforcement learning. *arXiv preprint arXiv:2406.00645*.
- Guo, H., Keyvan-Ekbatani, M., Xie, K., 2024. Modeling coupled driving behavior during lane change: A multi-agent transformer reinforcement learning approach. *Transportation Research Part C: Emerging Technologies* 165, 104703.
- Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International conference on machine learning*, PMLR. pp. 1861–1870.
- Han, X., Yang, Q., Chen, X., Cai, Z., Chu, X., Zhu, M., 2024. Autoreward: Closed-loop reward design with large language models for autonomous driving. *IEEE Transactions on Intelligent Vehicles*.
- Hart, F., Okhrin, O., Treiber, M., 2024. Towards robust car-following based on deep reinforcement learning. *Transportation research part C: emerging technologies* 159, 104486.
- Hazra, R., Sygkounas, A., Persson, A., Loutfi, A., Martires, P.Z.D., 2024. Revolve: Reward evolution with large language models for autonomous driving. *arXiv preprint arXiv:2406.01309*.
- He, X., Huang, W., Lv, C., 2024. Trustworthy autonomous driving via defense-aware robust reinforcement learning against worst-case observational perturbations. *Transportation Research Part C: Emerging Technologies* 163, 104632.
- Huang, Y., Sansom, J., Ma, Z., Gervits, F., Chai, J., 2024a. Drivlm: Enhancing llm-based autonomous driving agents with embodied and social experiences. *arXiv preprint arXiv:2406.03008*.
- Huang, Z., Chen, S., Pian, Y., Sheng, Z., Ahn, S., Noyce, D.A., 2024b. Toward c-v2x enabled connected transportation system: Rsu-based cooperative localization framework for autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems*.
- Huang, Z., Sheng, Z., Ma, C., Chen, S., 2024c. Human as ai mentor: Enhanced human-in-the-loop reinforcement learning for safe and efficient autonomous driving. *Communications in Transportation Research* 4, 100127.
- Huang, Z., Sheng, Z., Shi, L., Chen, S., 2024d. Trustworthy human-ai collaboration: Reinforcement learning with human feedback and physics knowledge for safe autonomous driving. *arXiv preprint arXiv:2409.00858*.
- Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al., 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.
- Knox, W.B., Allievi, A., Banzhaf, H., Schmitt, F., Stone, P., 2023. Reward (mis) design for autonomous driving. *Artificial Intelligence* 316, 103829.
- Kwon, M., Xie, S.M., Bullard, K., Sadigh, D., 2023. Reward design with language models. *arXiv preprint arXiv:2303.00001*.
- Leurent, E., 2018. An environment for autonomous driving decision-making. <https://github.com/eleurent/highway-env>.
- Ma, Y.J., Liang, W., Wang, G., Huang, D.A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., Anandkumar, A., 2023. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*.
- Mahmoudieh, P., Pathak, D., Darrell, T., 2022. Zero-shot reward specification via grounded natural language, in: *International Conference on Machine Learning*, PMLR. pp. 14743–14752.
- Mao, Z., Liu, Y., Qu, X., 2024. Integrating big data analytics in autonomous driving: An unsupervised hierarchical reinforcement learning approach. *Transportation Research Part C: Emerging Technologies* 162, 104606.
- Mei, J., Ma, Y., Yang, X., Wen, L., Cai, X., Li, X., Fu, D., Zhang, B., Cai, P., Dou, M., et al., 2024. Continuously learning, adapting, and improving: A dual-process approach to autonomous driving. *arXiv preprint arXiv:2405.15324*.
- OpenAI, 2023. Chatgpt. URL: <https://chat.openai.com/>.
- Polanyi, M., 2009. The tacit dimension, in: *Knowledge in organisations*. Routledge, pp. 135–146.
- Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al., 2021. Learning transferable visual models from natural language supervision, in: *International conference on machine learning*, PMLR. pp. 8748–8763.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N., 2021. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research* 22, 1–8.
- Rocamonde, J., Montesinos, V., Nava, E., Perez, E., Lindner, D., 2024. Vision-language models are zero-shot reward models for reinforcement learning.
- Schuhmann, C., Beaumont, R., Vencu, R., Gordon, C., Wightman, R., Cherti, M., Coombes, T., Katta, A., Mullis, C., Wortsman, M., et al., 2022. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in Neural Information Processing Systems* 35, 25278–25294.
- Shao, H., Hu, Y., Wang, L., Song, G., Waslander, S.L., Liu, Y., Li, H., 2024. Lmdrive: Closed-loop end-to-end driving with large language models, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15120–15130.
- Sheng, Z., Huang, Z., Chen, S., 2024a. Ego-planning-guided multi-graph convolutional network for heterogeneous agent trajectory prediction. *Computer-Aided Civil and Infrastructure Engineering* 39, 3357–3374.
- Sheng, Z., Huang, Z., Chen, S., 2024b. Traffic expertise meets residual rl: Knowledge-informed model-based residual reinforcement learning for cav trajectory control. *Communications in Transportation Research* 4, 100142.
- Sontakke, S., Zhang, J., Arnold, S., Pertsch, K., Biyik, E., Sadigh, D., Finn, C., Itti, L., 2024. Roboclip: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems* 36.
- Tian, X., Gu, J., Li, B., Liu, Y., Wang, Y., Zhao, Z., Zhan, K., Jia, P., Lang, X., Zhao, H., 2024. Drivelm: The convergence of autonomous driving and large vision-language models. *arXiv preprint arXiv:2402.12289*.
- Venuto, D., Islam, S.N., Klissarov, M., Precup, D., Yang, S., Anand, A., 2024. Code as reward: Empowering reinforcement learning with vlms. *arXiv preprint arXiv:2402.04764*.
- Wang, L., Liu, J., Shao, H., Wang, W., Chen, R., Liu, Y., Waslander, S.L., 2023a. Efficient reinforcement learning for autonomous driving with parameterized skills and priors, in: *Robotics: Science and Systems (RSS 2023)*.
- Wang, W., Xie, J., Hu, C., Zou, H., Fan, J., Tong, W., Wen, Y., Wu, S., Deng, H., Li, Z., et al., 2023b. Drivelm: Aligning multi-modal large language models with behavioral planning states for autonomous driving. *arXiv preprint arXiv:2312.09245*.
- Wang, Y., Sun, Z., Zhang, J., Xian, Z., Biyik, E., Held, D., Erickson, Z., 2024. Rl-vlm-f: Reinforcement learning from vision language foundation model feedback. *arXiv preprint arXiv:2402.03681*.
- Wei, Y., Wang, Z., Lu, Y., Xu, C., Liu, C., Zhao, H., Chen, S., Wang, Y., 2024. Editable scene simulation for autonomous driving via collaborative llm-agents, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15077–15087.
- Wen, L., Fu, D., Li, X., Cai, X., Ma, T., Cai, P., Dou, M., Shi, B., He, L., Qiao, Y., 2023. Dilu: A knowledge-driven approach to autonomous driving with large language models. *arXiv preprint arXiv:2309.16292*.
- Wu, J., Huang, C., Huang, H., Lv, C., Wang, Y., Wang, F.Y., 2024. Recent advances in reinforcement learning-based autonomous driving behavior planning: A survey. *Transportation Research Part C: Emerging Technologies* 164, 104654.
- Xie, T., Zhao, S., Wu, C.H., Liu, Y., Luo, Q., Zhong, V., Yang, Y., Yu, T., 2024. Text2reward: Reward shaping with language models for reinforcement learning, in: *The Twelfth International Conference on Learning Representations*.
- Xu, Z., Zhang, Y., Xie, E., Zhao, Z., Guo, Y., Wong, K.Y.K., Li, Z., Zhao, H., 2024. Drivegpt4: Interpretable end-to-end autonomous driving via large language model. *IEEE Robotics and Automation Letters*.
- Yao, Z., Li, X., Lang, B., Chuah, M.C., 2023. Goal-lbp: Goal-based local behavior guided trajectory prediction for autonomous driving. *IEEE Transactions on Intelligent Transportation Systems*.

- Ye, X., Tao, F., Mallik, A., Yaman, B., Ren, L., 2024. Lord: Large models based opposite reward design for autonomous driving. arXiv preprint arXiv:2403.18965 .
- Yildirim, M., Dagda, B., Fallah, S., 2024. Highwayllm: Decision-making and navigation in highway driving with rl-informed language model. arXiv preprint arXiv:2405.13547 .
- You, J., Shi, H., Jiang, Z., Huang, Z., Gan, R., Wu, K., Cheng, X., Li, X., Ran, B., 2024. V2x-vlm: End-to-end v2x cooperative autonomous driving through large vision-language models. arXiv preprint arXiv:2408.09251 .
- Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.H., Arenas, M.G., Chiang, H.T.L., Erez, T., Hasenclever, L., Humplik, J., et al., 2023. Language to rewards for robotic skill synthesis. arXiv preprint arXiv:2306.08647 .
- Zhang, J., Xu, C., Li, B., 2024. Chatscene: Knowledge-enabled safety-critical scenario generation for autonomous vehicles, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 15459–15469.
- Zhou, Z., Zhang, J., Zhang, J., Wang, B., Shi, T., Khamis, A., 2024. In-context learning for automated driving scenarios. arXiv preprint arXiv:2405.04135 .